

Grupo 3: Floricultura

Nome	E-mail
Amanda Lemos Ribas,	amandaribas@ic.uff.br
Isabella Direito Labre Martins,	isamartins@id.uff.br
Juliana Alves Poustka,	julianapoustka@id.uff.br
Lais Ferreira Nazareth,	laisfn@id.uff.br
Luiza Canto Furley Schmidt,	lufurley@id.uff.br
Maria Eduarda D' Angelo Quitete Vianna.	me_vianna@id.uff.br

Histórico de Revisão

Data	Autor	Descrição	Versão
29/03/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo	Reunião inicial para o estabelecimento do tema do projeto, definição dos requisitos não funcionais, funcionais e os principais casos de uso	0.0.1
04/04/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo	Desenvolvimento dos diagramas de caso de uso e formatação do documento.	0.0.2
11/04/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo	Definição dos requisitos arquiteturais, representação arquitetural (definição dos padrões arquiteturais e justificativa das decisões arquiteturais)	0.0.3
11/04/2026	Amanda Lemos,	Desenho do diagrama de	0.0.4

Grupo 3: Floricultura

Data	Autor	Descrição	Versão
	Isabella Direito, Maria Eduarda D'Angelo	classes inicial, voltado a nível de projeto.	
11/04/2026	Lais Ferreira Nazareth, Luiza Canto, Juliana Alves	Desenho de diagrama de visão arquitetural.	0.05
20/04/2026	Amanda Lemos Ribas	Atualizações no Diagrama de Classes a Nível Projeto e Modelo Conceitual.	0.0.6
23/04/2026	Isabella Direito, Maria Eduarda D'Angelo	Ajuste nas cardinalidades e nomenclaturas do diagrama de classes de projeto. Desenho do diagrama do modelo conceitual	0.0.7
24/04/2026	Amanda Lemos Ribas, Isabella Direito, Luiza Canto, Maria Eduarda D'Angelo,	Finalização do Documento, inclusão de diagramas feitos separadamente, commits no github	0.0.8
25/04/2026	Lais Ferreira Nazareth Luiza Canto Juliana Alves	Revisão final dos diagramas de Visão Arquitetural	0.0.9
02/05/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo	Ajustes após o feedback da apresentação: Revisão dos diagramas de sequência atualizados individualmente e atualização do diagrama de classes e do modelo conceitual	0.0.10
06/05/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto	Reunião final antes da entrega 1, verificando os diagramas feitos individualmente e atualização final no de classes.	0.0.11
05/06/2026	Amanda Lemos, Isabella Direito, Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo	Documentação dos Padrões GRASP e SOLID utilizados no projeto.	0.0.12
12/06/2026	Amanda Lemos, Isabella Direito,	Documentação da implementação com os frameworks e tecnologias utilizadas	0.0.13

Grupo 3: Floricultura

Data	Autor	Descrição	Versão
	Juliana Alves, Lais Ferreira, Luiza Canto, Maria Eduarda D'Angelo		

Sumário

1. [INTRODUÇÃO](#)
 - [1.1 Finalidade](#)
 - [1.2 Escopo](#)
2. [REPRESENTAÇÃO ARQUITETURAL](#)
 - [2.1 Definição dos padrões arquiteturais adotados](#)
 - [2.2 Justificativa das decisões arquiteturais](#)
 - [2.3 Diagrama de Visão Arquitetural](#)
3. [REQUISITOS E RESTRIÇÕES ARQUITETURAIS](#)
 - [3.1 Requisitos Funcionais](#)
 - [3.2 Requisitos Não Funcionais](#)
 - [3.3 Requisitos Arquiteturais, objetivos e restrições](#)
4. [VISÃO DE CASOS DE USO](#)
 - [4.1 Casos de Uso significantes para a arquitetura](#)
5. [VISÃO LÓGICA](#)
 - [5.1 Modelo Conceitual](#)
6. [VISÃO DE IMPLEMENTAÇÃO](#)
 - [6.1 Diagrama de Classes de Projeto](#)
 - [6.2 Diagrama de Sequência](#)
 - [6.3 Diagrama de Interação](#)
7. [APLICAÇÃO DE PRINCÍPIOS E PADRÕES](#)
 - [7.1 Controlador \(GRASP\)](#)
 - [7.2 Criador \(GRASP\)](#)
 - [7.3 Princípio de Aberto/Fechado \(SOLID\)](#)
 - [7.4 Substituição de Liskov \(SOLID\)](#)
 - [7.5 Inversão de Dependência \(SOLID\)](#)
8. [PADRÕES GOF](#)
 - [8.1 Factory Method para Produtos](#)
 - [8.2 Observer para Favoritar Produto](#)
 - [8.3 Strategy para Precificação dos Planos de Assinatura](#)

Grupo 3: Floricultura

[8.4 State para o ciclo de vida do pedido](#)

[8.5 Facade para Dashboard](#)

[8.6 Chain of Responsibility para validação do Pedido](#)

9. [LINHA DE PRODUTO DE SOFTWARE \(LPS\)](#)

10. [DESCRIÇÃO DE IMPLEMENTAÇÃO](#)

[10.1 Implementações do sistema](#)

[10.2 Tecnologias, frameworks e ferramentas utilizadas](#)

[10.3 Arquitetura geral da solução](#)

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente de um sistema de Floriculturas, usando diversas visões de arquitetura para representar diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.

1.2 Escopo

O objetivo do sistema é oferecer uma plataforma web para a floricultura “Tudo são flores”, permitindo a venda de todos seus produtos, como kits personalizados fixos, kits personalizados para datas comemorativas, flores individuais, arranjos, buquês, flores secas e acessórios, bem como assinaturas quinzenais com flores exclusivas aos assinantes.

Além das vendas diretas, o sistema atuará na automação logística, gerindo o ciclo de vida das assinaturas quinzenais e a reserva de materiais para eventos.

Os clientes poderão se cadastrar no site para aproveitar cupons de desconto, assinar o plano quinzenal, adicionar e manter itens no carrinho e realizar compras. Enquanto os usuários administradores, com seu cadastro próprio, poderão gerenciar o cadastro de produtos, com controle de estoque e garantir o gerenciamento das regras promocionais.

2. REPRESENTAÇÃO ARQUITETURAL

2.1. Definição dos padrões arquiteturais adotados

O sistema utiliza Arquitetura em Camadas (Layered Architecture) para organizar as responsabilidades do software e o padrão MVC (Model-View-Controller) para estruturar a lógica interna do Back-end.

- Arquitetura em Camadas: O projeto é dividido em três níveis principais para garantir o desacoplamento. A Camada de Apresentação (implementada em React) lida com a interface; a Camada de Negócio (Spring Boot Services) processa as regras da floricultura e é a base do Back-End; por último, a Camadas de Dados (MySQL) garante a persistência das informações.

Grupo 3: Floricultura

- **Padrão MVC no Back-end:** Dentro do ecossistema Spring, adotamos o MVC para organizar o fluxo de dados:
 - **Model:** Representado pelas entidades JPA que mapeiam as tabelas do banco de dados MySQL (ex: Produto, Pedido).
 - **Controller:** Responsável por expor os endpoints da API que o React consome e direcionar as requisições para os serviços corretos.
 - **View:** Como o Back-end atua como um provedor de serviços, a "View" tradicional é substituída pela Representação de Estado em formato JSON. O Spring Boot converte os objetos do Model em JSON, que é consumido pelo React para renderizar a interface final ao usuário.

2.2 Justificativa das decisões arquiteturais

Dentre as justificativas das decisões arquiteturais, podemos destacar:

- **Manutenibilidade (RNF-8):** A separação em camadas permite que uma alteração na interface (React) não exija mudanças na lógica de cálculo de estoque ou no banco de dados. O uso de JPA/Hibernate facilita a manutenção do banco de dados, pois o código Java fica independente de comandos SQL específicos.
- **Consistência de Dados (RA-9):** O uso do MySQL como banco de dados relacional, combinado com o gerenciamento transacional do Spring, garante que pedidos e baixas de estoque sejam processados de forma íntegra, evitando estados inconsistentes no sistema.
- **Produtividade e Padronização (RA-1):** O MVC é um padrão amplamente documentado e suportado pelo Spring Boot. Isso facilita a implementação de novas funcionalidades, como o sistema de assinaturas ou cupons, seguindo uma estrutura de pastas e responsabilidades já conhecida pela equipe.
- **Portabilidade (RNF-5):** Como a lógica de negócio está isolada na camada de serviço do Spring e o banco é acessado via JPA, o sistema pode ser facilmente migrado entre diferentes servidores ou sistemas operacionais sem perda de funcionalidade.

2.3. Diagrama de Visão Arquitetural

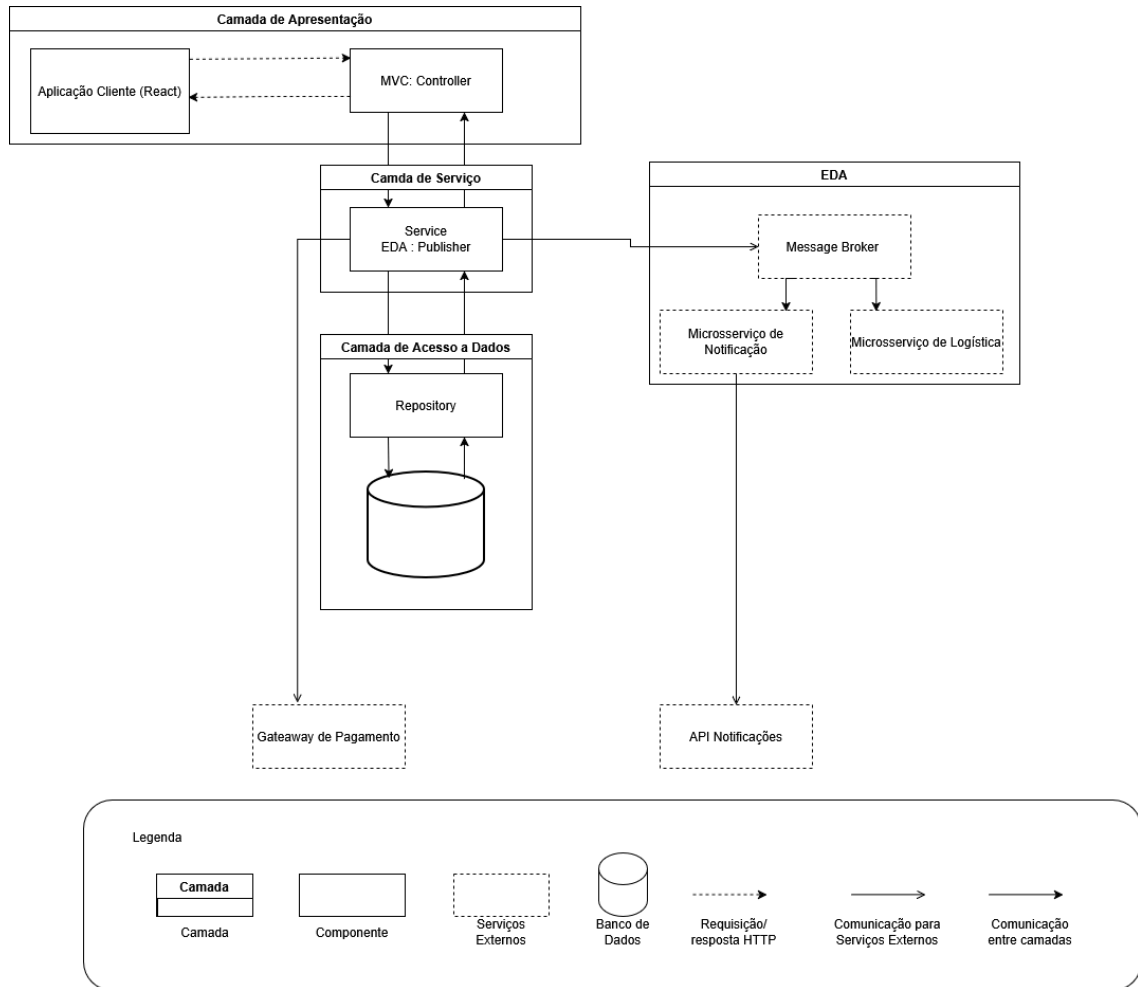


Imagem 1 - Diagrama de Visão Arquitetural.

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e restrições que têm um impacto significativo na arquitetura.

3.1. Requisitos Funcionais:

- **Cadastro do Cliente**

RF-1: O sistema deve permitir que o usuário se cadastre no site por meio da conta Google, além de informar o CPF e telefone.

RF-2: O sistema deve possuir uma tela de login para que usuários cadastrados acessem sua conta por meio da conexão com a conta google.

RF-3: O sistema deve permitir que o usuário edite seu perfil e altere dados cadastrais (com exceção do CPF).

Grupo 3: Floricultura

RF-4: O sistema deve permitir cadastros por CNPJ, para empresas.

- **Catálogo de produtos**

RF-5: O sistema deve exibir um catálogo de produtos contendo arranjos prontos e acessórios (cartões personalizados, chocolates).

RF-6: O sistema deve permitir que o usuário clique em um produto para visualizar detalhes (como a quantidade, cor e tipos de flores em um buquê)

RF-7: O sistema deve permitir a organização dos produtos por categorias (flores secas, buquês prontos e kits comemorativos por exemplo)

RF-8: O sistema deve permitir a busca e filtragem de produtos por categoria, preço, tipo de evento.

RF-9: O sistema deve reservar a quantidade de produtos referentes a assinatura quinzenal

RF-10: O sistema deve permitir a reserva de datas e materiais para decoração de festas e casamentos.

- **Compra**

RF-11: O sistema deve oferecer planos de assinatura para que o cliente receba flores em intervalos quinzenais, com flores exclusivas aos assinantes.

RF-12:: O sistema deve permitir que o usuário escolha a opção de assinatura e o tipo de flor (buquê, vaso ou arranjo).

RF-13: O sistema deve permitir a inclusão de produtos a um carrinho virtual, informando a quantidade desejada

RF-14: O sistema deve permitir a remoção de produtos do carrinho

RF-15: O sistema deve permitir alterar a quantidade de um produto no carrinho

RF-16: O sistema deve exibir os itens do carrinho, com quantidade, preços e subtotal

RF-17: O sistema deve manter os itens nos carrinhos de usuários cadastrados, a menos que o próprio usuário remova os produtos

RF-18: O sistema deve permitir que um cliente efetue uma compra a partir dos itens no carrinho, realizando uma validação de disponibilidade no estoque

Grupo 3: Floricultura

RF-19: O sistema deve permitir que o usuário informe ou selecione (dentre seus endereços cadastrados) um endereço de entrega para a compra

RF-20: O sistema deve calcular o valor do frete com base na distância do cliente até a unidade mais próxima.

RF-21: O sistema deve permitir que o cliente aplique cupons de desconto válidos no carrinho de compras, recalculando o valor total do pedido automaticamente.

RF-22: O sistema deve exibir um resumo da compra contendo itens, frete, descontos e valor total antes de prosseguir para pagamento

RF-23: O sistema deve permitir que o usuário tenha acesso a uma lista com histórico de pedidos.

RF-24: O sistema deve permitir a realização de reservas.

RF-25: O sistema deve enviar avisos automáticos (via e-mail ou Whatsapp) sobre confirmação de compra e status do pedido mudar para “Enviado”.

RF-26: O sistema deve ser capaz de processar diversas formas de pagamento, incluindo cartões de crédito, débito, pix, etc.

RF-27: O sistema deve permitir que cada usuário cadastre até 5 cartões.

RF-28: O sistema deve gerar uma lista com todas as saídas previstas para o dia (vendas, assinaturas e eventos).

RF-29: O sistema deve permitir que o cliente atribua um comentário a um pedido após a entrega.

RF-30: O sistema deve permitir que o cliente monte um buquê personalizado, selecionando flores, quantidades, embalagens e adicionais.

- **Gerenciamento Interno:**

RF-31: O sistema deve garantir que os usuários administradores gerenciem o estoque.

RF-32: O sistema deve manter um histórico de atualização dos usuários administradores, permitindo controle das mudanças do sistema.

RF-33: O sistema deve permitir a inclusão (cadastro) de novos produtos a qualquer momento, bem como a remoção dos mesmos.

Grupo 3: Floricultura

RF-34: O sistema deve permitir que o administrador da floricultura visualize e responda às avaliações dos clientes.

RF-35: O sistema deve permitir que o administrador cadastre, edite e remova cupons de desconto e regras de promoção com períodos de validade específicos.

RF-36: O sistema deve permitir o registro de entrada de flores no estoque, informando quantidade e tipo de controle (unidade ou maço)

RF-37: O sistema deve atualizar automaticamente o estoque ao realizar uma venda

RF-38: O sistema deve permitir o cadastro de flores secas, que possuem controle de estoque por lote e não possuem data de validade como as naturais.

RF-39: O sistema deve possuir integração com o banco de dados para controle de estoque.

RF-40: O sistema deve exibir um catálogo de imagens de eventos anteriores para que os clientes selecionem o estilo de decoração que deseja contratar.

RF-41: O sistema deve validar a vigência e as condições de uso (valor mínimo, categoria e limite de usos) de um cupom no momento da finalização da compra, notificando o usuário caso o benefício não seja mais aplicável.

RF-42: O sistema deve permitir que o administrador defina se uma campanha é cumulativa com outras promoções vigentes.

RF-43: O sistema deve permitir a criação de cupons específicos para o custo do frete (Frete Grátis).

3.2. Requisitos Não Funcionais:

RNF-1: Disponibilidade

DI-1: A funcionalidade do sistema deve manter padrão por 99.9% do tempo em dias úteis e horários de movimento padrão.

DI-2: Em épocas de pico, como datas comemorativas, o sistema deve manter funcionalidade acima de 99.9% do tempo.

RNF-2: Eficiência de Desempenho

ED-1: O sistema deve apresentar tempo de resposta inferior a 2 segundos, considerando condições normais de operação.

Grupo 3: Floricultura

ED-2: O sistema deve suportar acesso de múltiplos usuários simultaneamente, sem quedas de desempenho.

RNF-3: Escalabilidade

E-1: Em épocas de pico, datas comemorativas, o sistema deve crescer horizontalmente, sendo capaz de aumentar de carga tanto na máquina servidor existentes, quanto adicionando mais máquinas (nós) para suprir o fluxo de requisições.

RNF-4: Segurança

SEG-1: O sistema deve garantir que os dados do usuário não sejam acessados ou divulgados por terceiros, assegurando o isolamento de dados entre os usuários.

SEG-2: O sistema deve garantir que o acesso seja realizado através de login e senha, com suporte a políticas de senha forte.

RNF-5: Portabilidade

POR-1: O sistema deve ser compatível com diferentes sistemas operacionais, como Linux, Windows, Mac.

POR-2: A interface web deve ser responsiva ao se adequar às proporções de dispositivos com telas e resoluções distintas.

RNF-6: Usabilidade

US-1: A interface do sistema deve ser intuitiva, exigindo no máximo 10 minutos para um usuário se familiarizar com a interface.

US-2: O sistema deve apresentar mensagens de erro em linguagem clara e não técnica, orientando o usuário sobre como corrigir o problema.

US-3: Caso o usuário cometa um erro em algum formulário, os dados corretos permanecem preenchidos, exigindo somente alteração dos erros.

RNF-7: Compatibilidade

COM-1: O sistema deve integrar-se com serviços externos de pagamento e entrega, garantindo o recebimento e envio correto das informações.

RNF-8: Manutenibilidade

MAN-1: O sistema deve ser estruturado de forma modular, permitindo manutenção independente de cada módulo.

Grupo 3: Floricultura

MAN-2: O sistema deve permitir a implementação de alterações com baixo impacto em outros módulos, minimizando risco e tempo de desempenho.

MAN-3: O sistema deve permitir a realização de testes, para garantir que as funcionalidades funcionem corretamente após as mudanças.

3.3 Requisitos Arquiteturais, objetivos e restrições

- **Módulos e Interfaces:**

RA-1: O sistema deve possuir uma separação clara entre lógica de negócio, acesso a dados e exposição de serviços para garantir a manutenibilidade.

Decisão: O back-end deve seguir o padrão Spring-Boot MVC (Model-View-Controller). Com as responsabilidades divididas em Controller, Service e Repository.

RA-2: O Sistema deve oferecer uma interface gráfica intuitiva que se adapte a diferentes tipos de usuários.

Decisões: O front-end será desenvolvido em React, utilizando Tailwind CSS para garantir que a ferramenta seja ajustada a diferentes perfis.

RA-3: O sistema deve permitir alterações visuais rápidas com baixo impacto no código-fonte, visando a evolução contínua da interface.

Decisão: Adoção do Tailwind CSS que elimina a necessidade de arquivos CSS gigantescos e complexos, permitindo que a manutenção da identidade visual seja feita diretamente nos componentes React.

RA-4: O sistema deve ser capaz de enviar notificações em tempo real e gerenciar o envio de mensagens automáticas sem comprometer a performance da página web.

Decisão: Utilização do Firebase (Firebase Cloud Messaging e Authentication) integrado ao front-end em React, visando reduzir a complexidade da infraestrutura própria e garantir alta disponibilidade nos serviços de comunicação e segurança.

RA-5: O sistema deve garantir que a interface web e o servidor se comuniquem de forma segura e eficiente.

Decisão: Uso de uma API REST como ponte de integração. O React consumirá os serviços do Spring Boot através de requisições HTTP, utilizando o formato JSON para a troca de informações entre front-end e back-end, o que assegura total independência entre as tecnologias.

Grupo 3: Floricultura

- **Segurança e autenticação:**

RA-6 : Integração de acesso simplificado utilizando contas existentes do Google para identificação do usuário.

Decisão: Implementação do Firebase Authentication com provedor de identidade do Google. O token gerado será validado pelo Back-end para garantir o acesso seguro aos dados do cliente.

RA-7: O sistema deve garantir a diferenciação técnica entre permissões de usuários clientes e usuários administradores

Decisão: Utilização do Spring Security para validar as permissões contidas no token do usuário.

RA- 8: Filtro rigoroso de todas as entradas de dados enviadas pelo usuário através de formulários.

Decisão: O Back-end em Spring Boot utilizará o mapeamento via JPA (Hibernate) para evitar consultas SQL manuais vulneráveis e aplicará anotações de validação para garantir que os dados recebidos sigam o formato esperado.

RA-9: O sistema deve garantir que os dados de produtos, clientes e pedidos sejam armazenados de forma consistente e segura.

Decisão: Criação de uma camada de persistência em Java para centralizar o acesso ao banco de dados. Todas as operações de salvar e buscar dados serão feitas por essa camada, garantindo organização e integridade das informações.

RA-10: O sistema deve delegar o processamento de pagamentos para provedores externos (Gateway/PSPs)

Decisão: O sistema não processará nem armazenará dados sensíveis de pagamento (como números de cartão de crédito). Para isso, será utilizada uma integração via API com um provedor especializado. O fluxo de pagamento será feito através de um “Checkout Transparente” ou “Redirect”, onde o sistema apenas envia o valor e recebe uma notificação confirmando o status da transação.

- **Portabilidade**

RA-11 O sistema deve funcionar em diferentes sistemas operacionais (Windows, Linux e macOS) sem precisar mudar o código.

Decisão: Utilização da JVM (Java Virtual Machine). A aplicação será compilada em bytecode, permitindo que a lógica de negócio seja executada de forma consistente em qualquer infraestrutura que suporte o ambiente Java, assegurando a independência da plataforma.

- **Compatibilidade**

Grupo 3: Floricultura

RA-12: O sistema deve permitir integração com diferentes serviços externos de entrega para envio de pedidos.

Decisão: Criação de uma camada de integração usando interfaces em Java. Cada serviço de entrega será implementado separadamente, permitindo trocar ou adicionar transportadoras sem alterar o fluxo de pedidos e checkout.

- **Escalabilidade e Elasticidade**

RA-13: O sistema deve ser capaz de crescer horizontalmente para suportar o aumento de carga em datas comemorativas de pico.

Decisão: A aplicação será hospedada em uma plataforma de Cloud Computing, utilizando serviços de Load Balancer (para distribuir requisições entre instâncias da aplicação) e Auto Scaling (para criar novas instâncias automaticamente, quando a CPU atingir o limite estipulado)

- **Persistência e Confiabilidade:**

RA-14: O sistema deve garantir que, em caso de falha durante um processo complexo (como a finalização de um pedido), os dados não fiquem em estado inconsistente.

Decisão: Uso de Gerenciamento Transacional no Spring Boot. Através da anotação `@Transactional`, o sistema garante que uma operação (como baixar o estoque e gerar o pedido) só seja confirmada se todas as etapas forem concluídas com sucesso; caso contrário, a operação é desfeita (rollback).

- **Desempenho**

RA-15: O sistema deve processar tarefas pesadas ou comunicações externas sem bloquear a execução imediata das requisições do cliente.

Decisão: Utilização de processamento assíncrono pelo Spring Boot.

Grupo 3: Floricultura

4. VISÃO DE CASOS DE USO

4.1 Casos de Uso significantes para a arquitetura

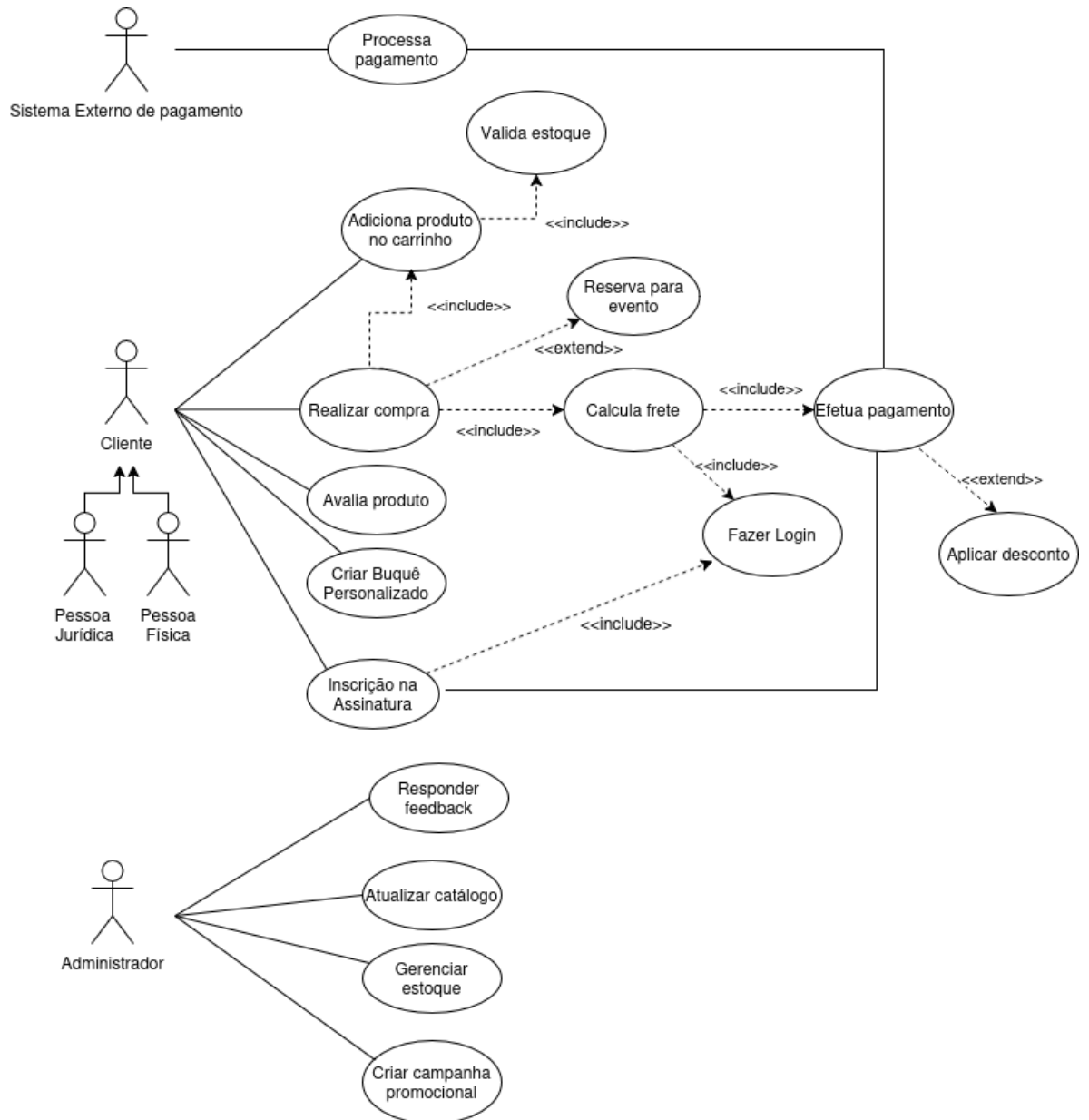


Imagem 2 – Diagrama de Caso de Uso das principais funcionalidades da floricultura.

5. VISÃO LÓGICA

5.1 Modelo Conceitual

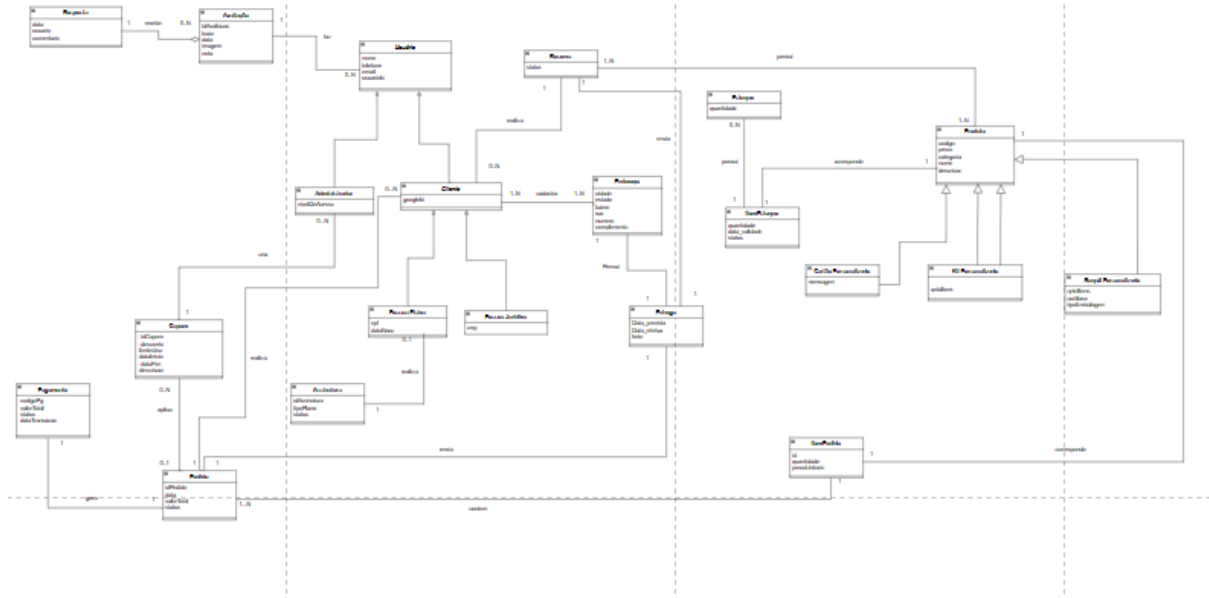


Imagem 3 – Modelo conceitual (Diagrama de Classes a nível de análise).

Grupo 3: Floricultura

6. VISÃO DE IMPLEMENTAÇÃO

6.1 Diagrama de Classes de Projeto

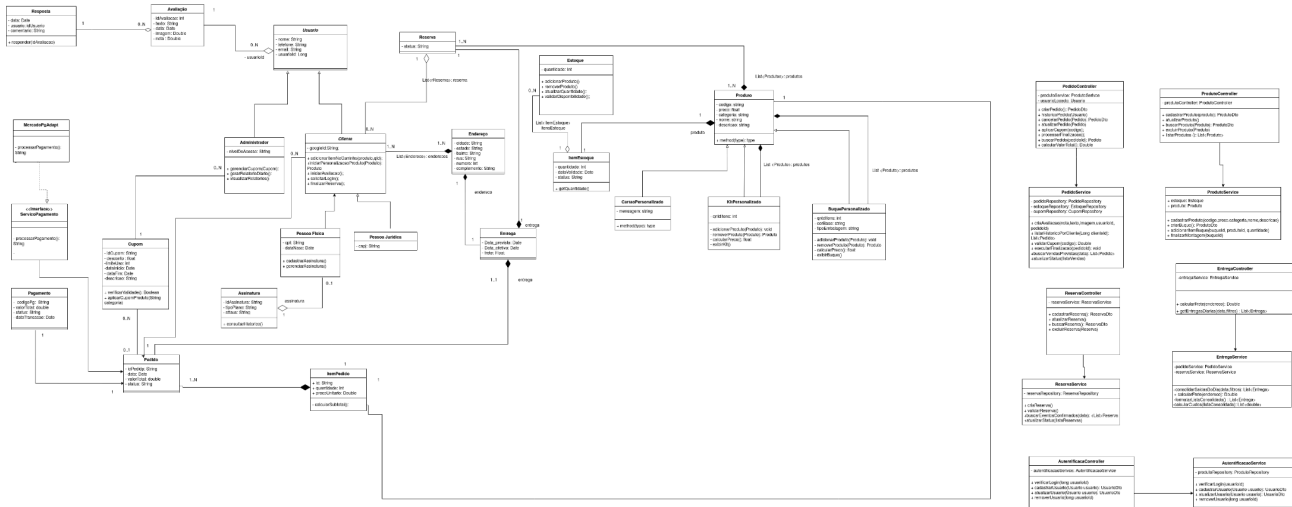


Imagem 4 – Diagrama de Classes à Nível Projeto.

Grupo 3: Floricultura

6.2 Diagrama de Sequência

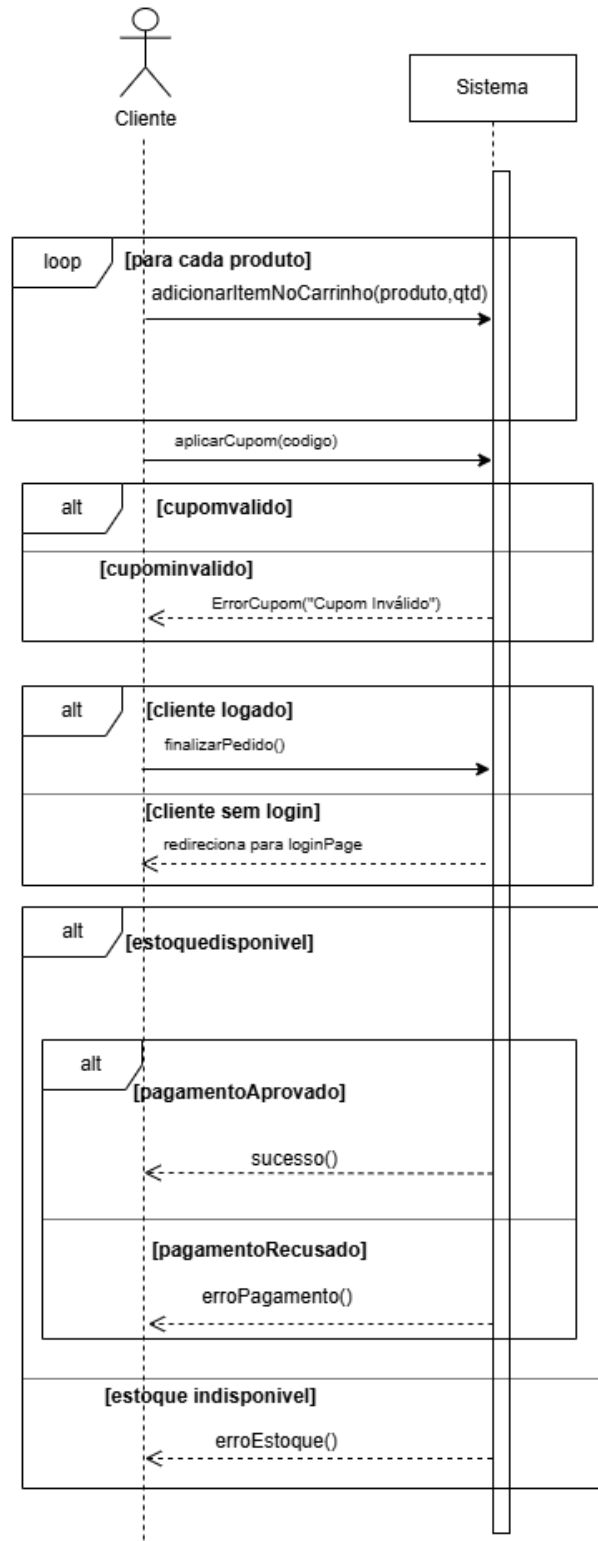


Imagem 5 - Diagrama de Sequência: UC-1 - Realizar Pedido

Grupo 3: Floricultura

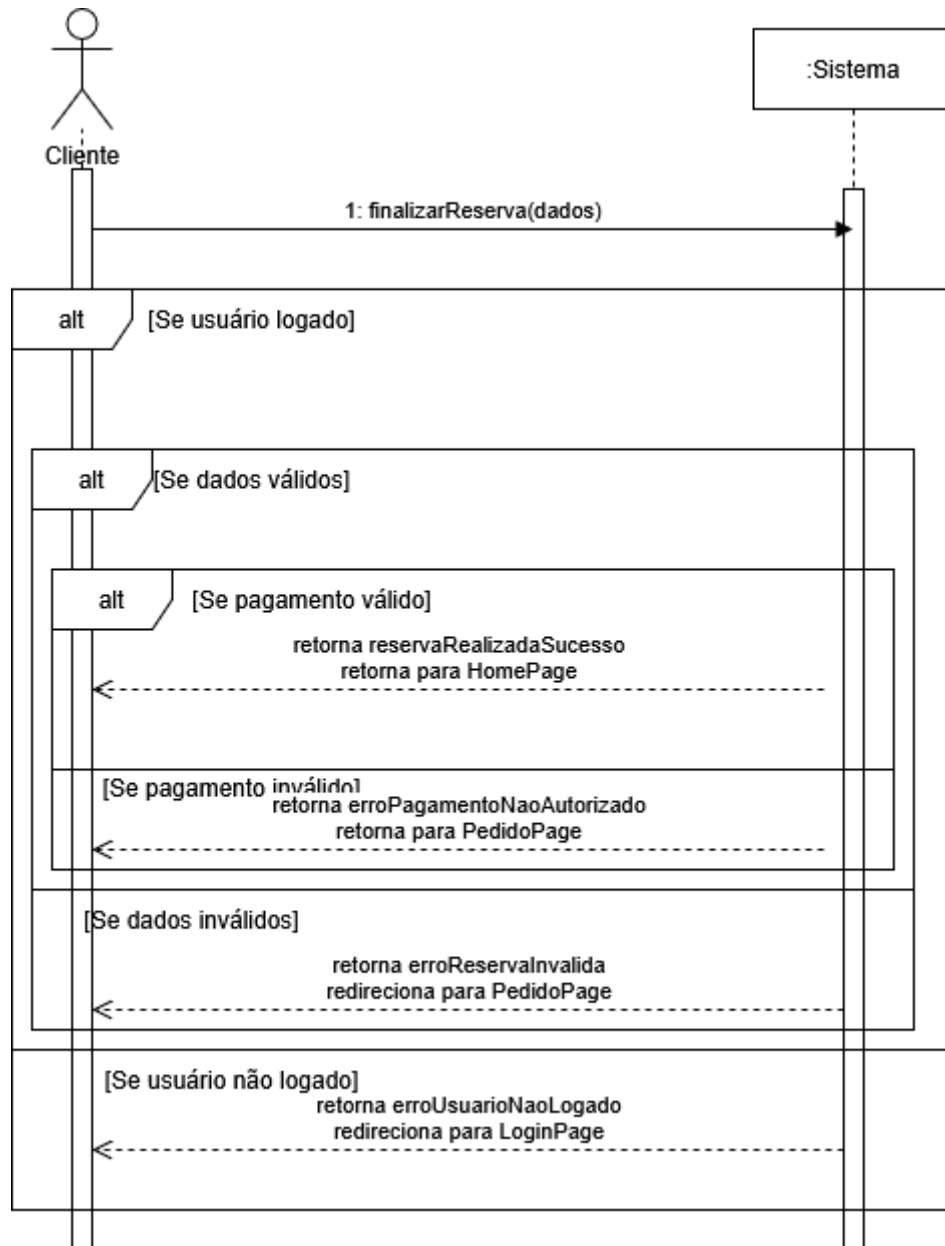


Imagem 6 - Diagrama de Sequência: UC-2 - Reservar Decorações para eventos

Grupo 3: Floricultura

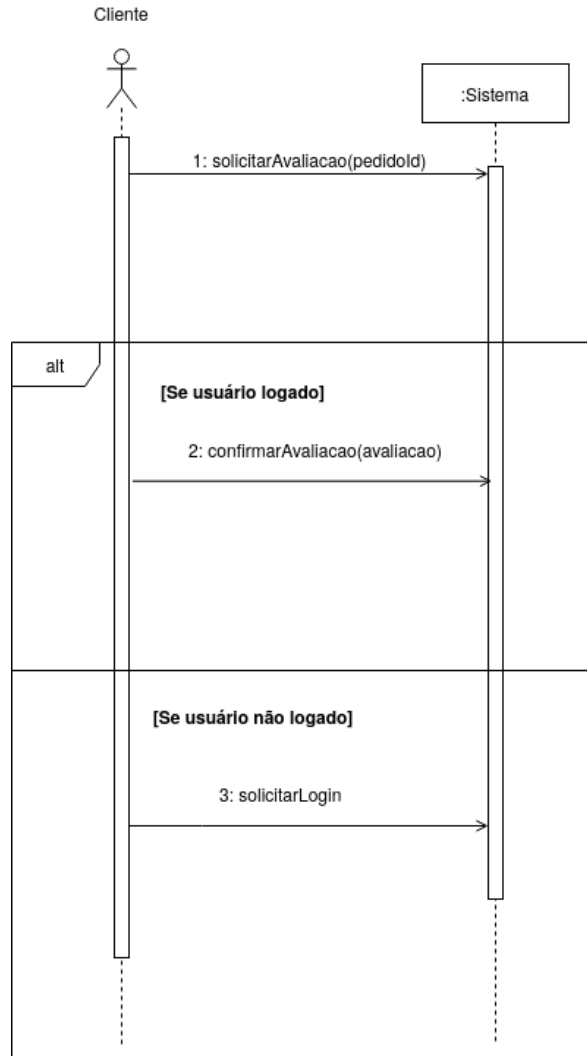


Imagem 7 - Diagrama de Sequência: UC-3 - Avaliar produto e gerir feedback

Grupo 3: Floricultura

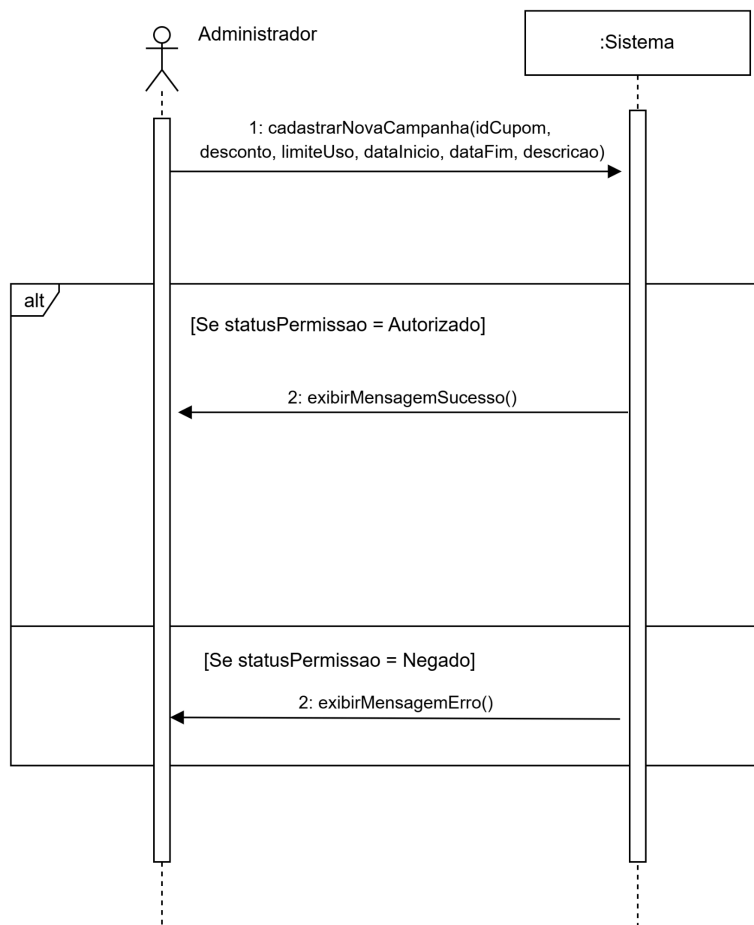


Imagem 8 - Diagrama de Sequência: UC-4 - Criar Campanha Promocional

Grupo 3: Floricultura

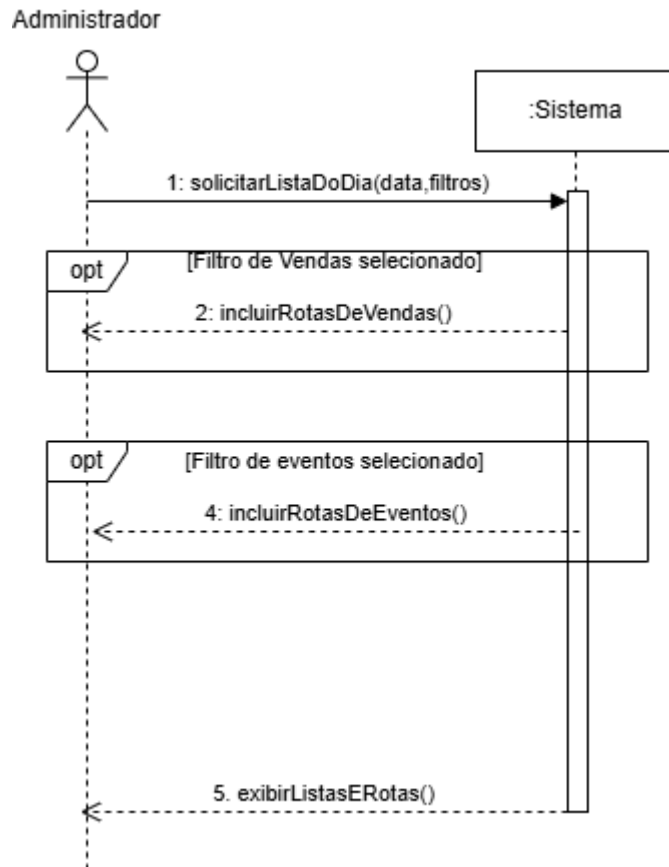


Imagem 9 - Diagrama de Sequência: UC 05 - Controle de entregas diárias

Grupo 3: Floricultura

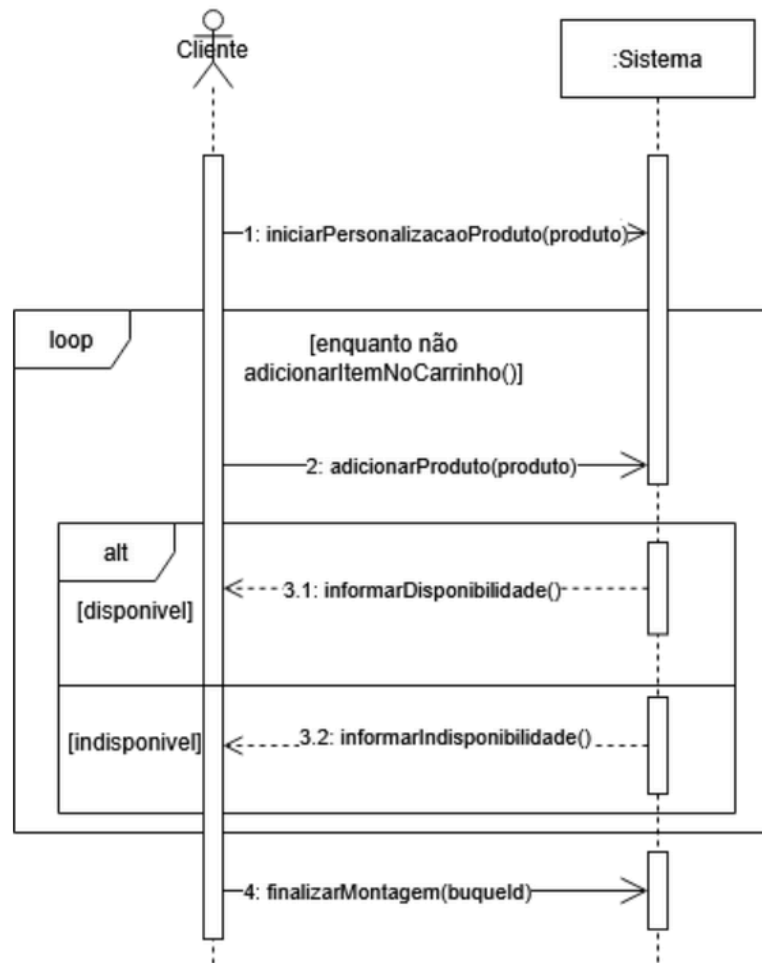


Imagem 10 - Diagrama de Sequência: UC-6 - Montar Buquê Personalizado

6.3 Diagrama de Interação

6.3.1 UC1 - Realizar Vendas

- Contrato da operação

Nome da operação: finalizarPedido

Parâmetros de entrada: itensCarrinho, enderecoEntrega

Referências Cruzadas: RF-18, RF-19, RF-20, RF-22, RF-25, RF-26, RF-36

Pré-Condições: Usuário deve estar autenticado no sistema, carrinho deve conter ao menos um item, endereço de entrega deve estar informado ou selecionado, produtos devem existir no sistema

Pós Condições:

em caso de sucesso: Atualização do estoque (baixa dos itens) realizada, pedido confirmado no sistema, associado ao cliente e armazenado no banco de dados, notificação de sucesso enviada ao cliente, valor do frete e total calculados e pagamento processado.

Grupo 3: Floricultura

em caso de fracasso: Nenhuma alteração no estoque do sistema e exibição de mensagem de erro de estoque ou de pagamento ao cliente.

- Diagrama de interação

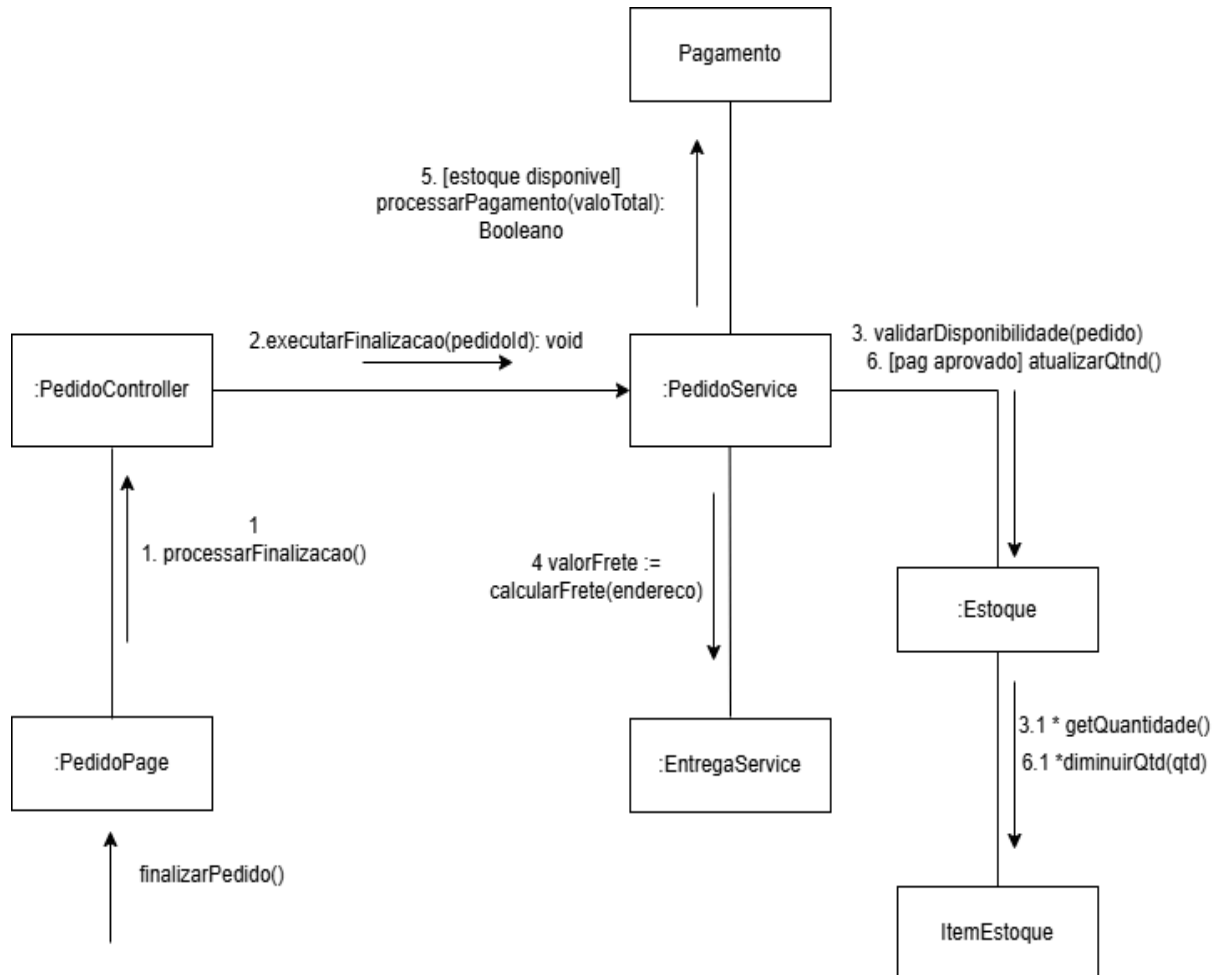


Imagem 11 – Diagrama de Interação (Comunicação) da operação finalizar pedido do caso de Uso 1 – Realizar vendas.

6.3.2 UC2 - Reservar Decorações para eventos

- Contrato da operação
Nome da operação: finalizarReserva
Parâmetros de entrada: itensCarrinho, dataReserva, enderecoEntrega
Referências Cruzadas: RF-2, RF-10, RF-18, RF-19, RF-24, RF-26, RF-37, RF-39
Pré-condições: Usuário deve estar autenticado no sistema, itens devem existir no estoque, carrinho deve ter ao menos um item, opção de entrega deve estar selecionada (entrega ou retirada).

Grupo 3: Floricultura

Pós-condições: Em caso de sucesso, estoque deve ser atualizado, a venda deve ser cadastrada no banco de dados, o pagamento deve ser validado e uma notificação de sucesso deve ser enviada.

Em caso de fracasso, a compra deve ser dada como não concluída e o erro específico deve ser informado ao cliente.t

- Diagrama de interação

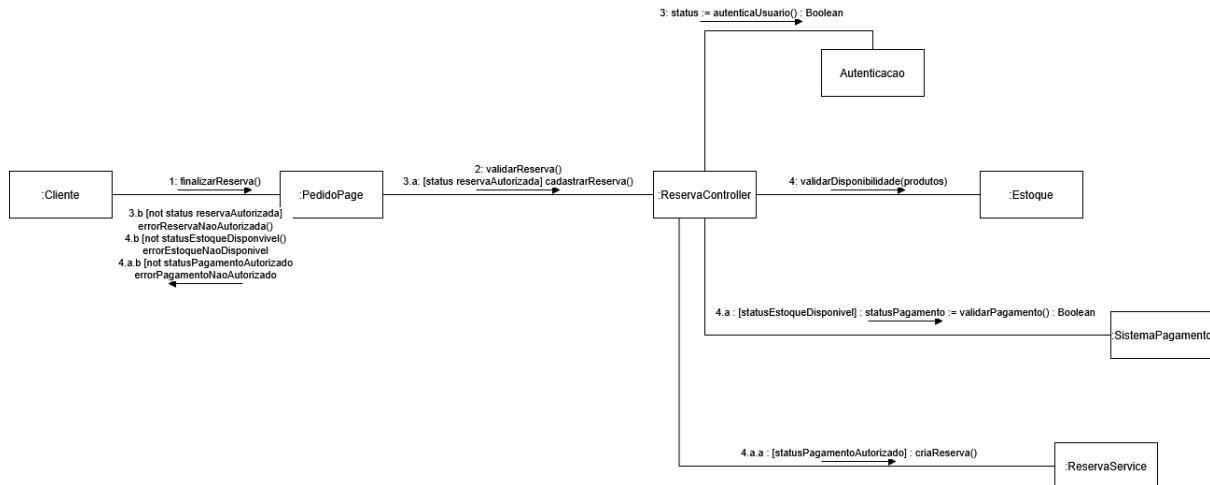


Imagem 12 – Diagrama de Interação (Comunicação) da operação finalizar reserva do caso de Uso 2 – Realizar decoração para eventos.

6.3.3 UC3 - Avaliar produto e gerir feedback

- Contrato da Operação
Nome da operação: submeter Avaliação;
Parâmetros de entrada: imagem e texto;
Referências Cruzadas: RF-29, UC-03: Avaliar produto e gerir feedback
Pré-Condições: Usuário logado, pedido e usuário existentes
Pós Condições: Criação do objeto comentário, associação entre comentário e pedido, armazenamento do comentário no sistema, comentário disponível para visualização.

- Diagrama de Interação

Grupo 3: Floricultura

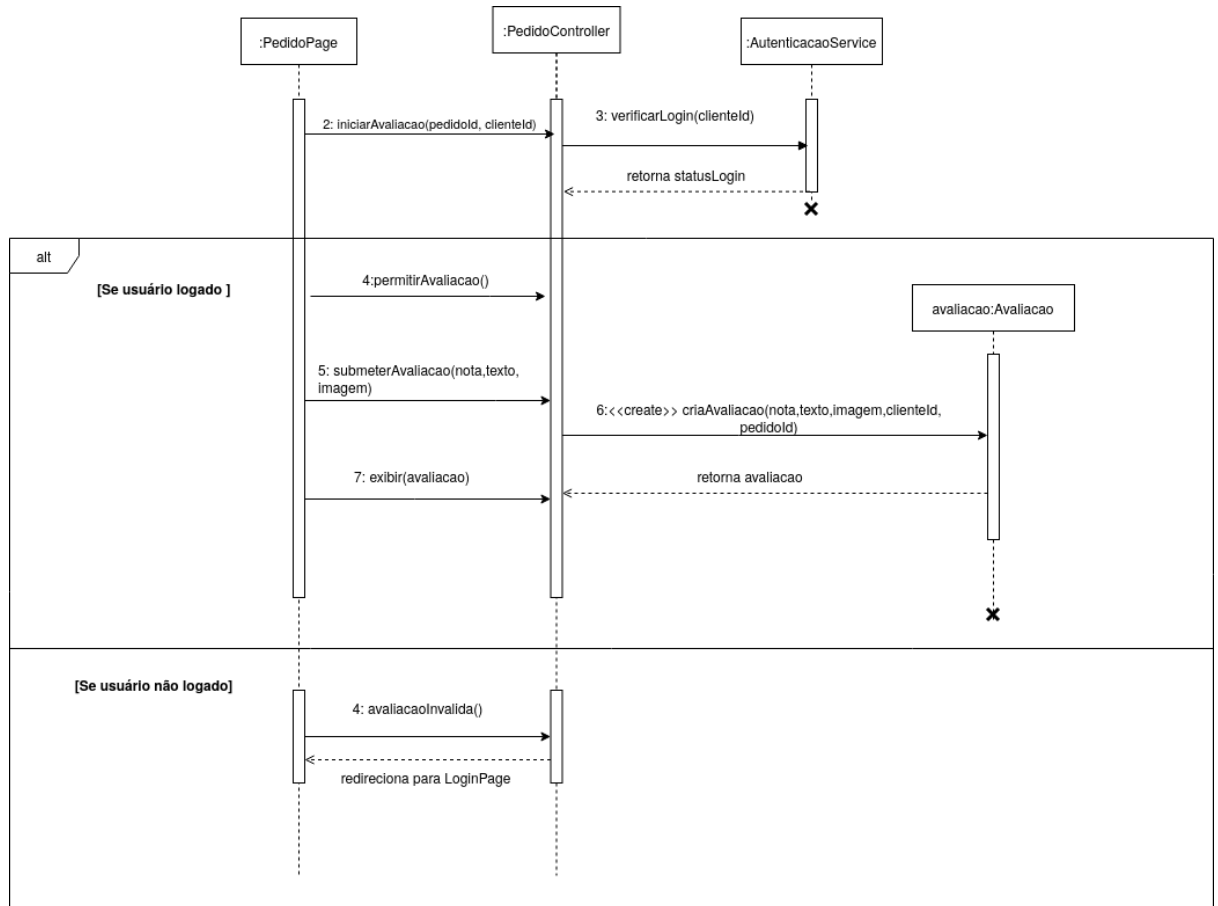


Imagem 13 - Diagrama de Sequência da operação submeter avaliação do Caso de uso 3 – Avaliar produto e gerir feedback.

6.3.4 UC4 - Criar Campanha promocional

- Contrato da operação

Nome da operação: Cadastrar Campanha

Parâmetros de entrada: Código, valor de desconto, tipo de promoção, data de início, data de fim, valor mínimo, limite de uso

Referências cruzadas: RF-21, RF-34, RF-40, RF-41, RF-42.

Pré-Condição: O administrador deve estar autenticado no sistema

Pós-Condição: Criação do objeto campanha

- Diagrama de Iteração

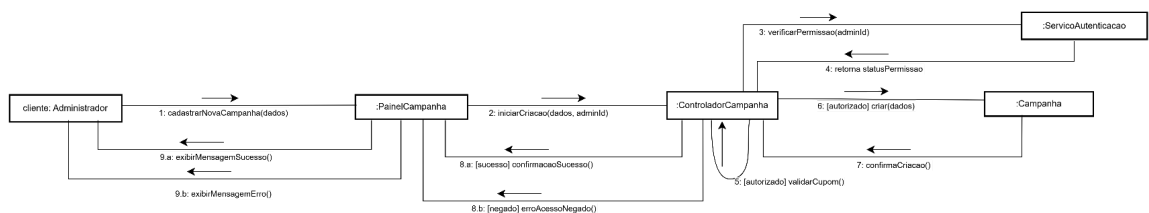


Imagem 13 - Diagrama de Interação (Comunicação) da operação cadastrar campanha do Caso de uso 4 – Criar campanha promocional

Grupo 3: Floricultura

6.3.5 UC5 - Controle de entregas diárias

- Contrato da operação
Nome da operação: solicitarListaDoDia
Parâmetros de entrada: data
Referências cruzadas: RF-28
Pré-Condição: O administrador deve estar autenticado no sistema
Pós-Condição: Uma lista consolidada contendo as saídas e entregas correspondentes à data solicitada é retornada pelo sistema e exibida na tela para acompanhamento logístico.

- Diagrama de interação

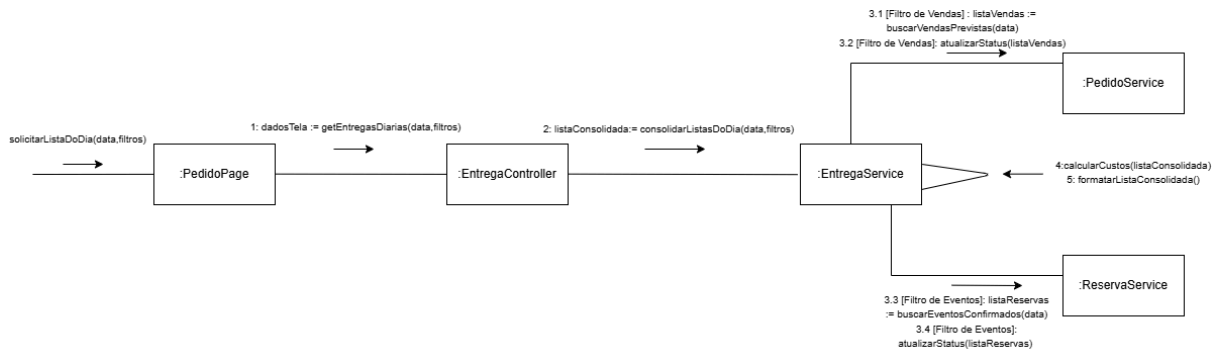


Imagem 14- Diagrama de Interação (Comunicação) da operação solicitar Lista do dia do Caso de Uso 5 – Controle de entregas diárias

6.3.6 UC6 - Montar buquê personalizado

- Contrato da operação
Nome da operação: Iniciar Personalização do Produto
Parâmetros de entrada: ID do produto
Referências cruzadas: RF-30
Pré-Condição: O cliente deve estar logado no sistema.
Pós-Condição: Buquê disponível para edição.
- Diagrama de interação

Grupo 3: Floricultura

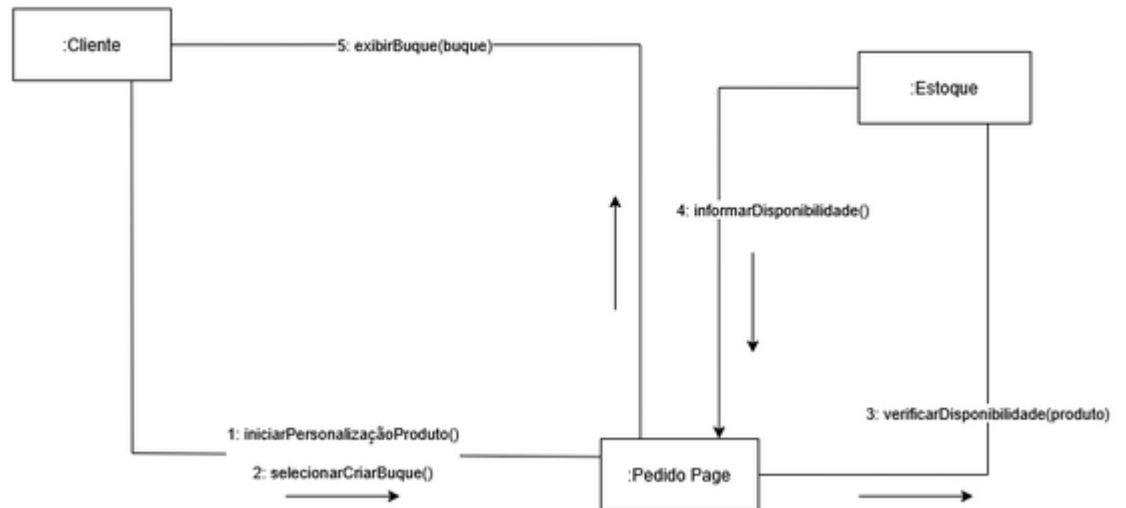
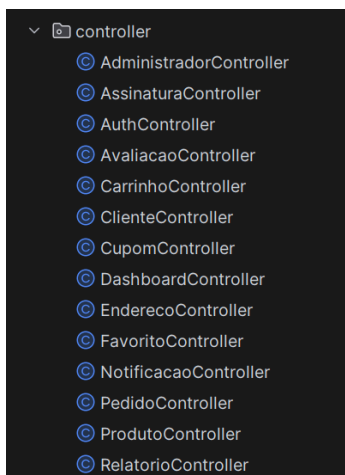


Imagem 15- Diagrama de Interação (Comunicação) da operação iniciar personalização do produto do Caso de Uso 6 – Montar buquê personalizado

7. APLICAÇÃO DE PRINCÍPIOS E PADRÕES

7.1 Controlador (GRASP)

O padrão GRASP Controlador é utilizado múltiplas vezes no projeto devido ao uso da arquitetura MVC (Model-View-Controller). Na parte do Spring-Boot, em cada CRUD e caso de uso, foi necessária a utilização de classes controladoras referentes aos modelos do sistema. As classes controladoras operam como intermediárias de baixo acoplamento. Não executam regras de negócio, mas sim interceptam as interações da camada de apresentação e delegam a execução para as estruturas de serviço (@Service), isolando a interface gráfica do modelo de dados.



```
public class ProdutoController {

    private final ProdutoService produtoService;

    // GET - Listar todos os produtos do sistema (Retorna uma lista mista de FlorDto,
    // KitDto, etc.)
    @GetMapping @Amanda Lemos Ribas
    public ResponseEntity<List<ProdutoDto>> recuperarProdutos() {
        List<ProdutoDto> produtos = produtoService.recuperarProdutos();
        return ResponseEntity.ok(produtos);
    }

    // GET - Filtrar produtos por categoria (Ex: /produtos/categoria/flores ou
    // /produtos/categoria/cartoes)
    @GetMapping("/{categoria}/{categoria}") @Amanda Lemos Ribas
    public ResponseEntity<List<ProdutoDto>> recuperarProdutosPorCategoria
        (@PathVariable String categoria) {
        List<ProdutoDto> produtos = produtoService.recuperarProdutosPorCategoria
            (categoria);
        return ResponseEntity.ok(produtos);
    }
}
```

Grupo 3: Floricultura

Imagens 16, 17 – Todos os controladores do sistema e exemplo de implementação de controlador

Dessa forma, o controlador possui um mapeamento dos métodos apresentados na camada de serviço.

7.2 Criador (GRASP)

O padrão Criador estabelece que a responsabilidade de criar um objeto deve ficar com a classe que possui as informações necessárias para isso ou que mantém uma relação direta com ele.

Essa abordagem ajuda a manter as responsabilidades bem definidas dentro do sistema. Cada classe passa a cuidar dos elementos que fazem parte do seu contexto. Com isso, o código fica mais organizado e fácil de manter, além de reduzir o impacto de mudanças futuras.

O Carrinho é o criador natural de ItemCarrinho, pois mantém uma relação de composição com ele. Em vez de deixar o serviço montar o item manualmente, a própria entidade Carrinho encapsula essa lógica. Método adicionarItem na entidade carrinho:

```
Java
@Entity
public class Carrinho {

    @OneToMany(mappedBy = "carrinho", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<ItemCarrinho> itens = new ArrayList<>();

    public void adicionarItem(Produto produto, Integer quantidade) {

        Optional<ItemCarrinho> existente = this.itens.stream()
            .filter(item ->
                item.getProduto().getCodigo().equals(produto.getCodigo()))
            .findFirst();

        if (existente.isPresent()) {
            existente.get().setQuantidade(existente.get().getQuantidade() +
                quantidade);
        } else {
            ItemCarrinho novoItem = new ItemCarrinho();
            novoItem.setProduto(produto);
            novoItem.setQuantidade(quantidade);
            novoItem.setCarrinho(this);           this.itens.add(novoItem);
        }
    }
}
```

Tabela 1 - Criação de ItemCarrinho pela entidade Carrinho

7.3 Princípio Aberto/Fechado (SOLID)

O princípio SOLID aberto-fechado consiste quando entidades do sistema estão abertas para extensão e fechadas para modificação. Dessa forma, o sistema segue o princípio a partir da implementação do Factory Method em produtos, que permite que novas classes herdadas de produtos sejam adicionadas ao sistema, bem como suas fábricas, sem precisar alterar a estrutura do projeto.

Inicialmente, a classe Produto consistia em somente uma classe concreta, em que todos os produtos compartilhavam os mesmos atributos, sendo diferenciados apenas pelos seus valores de atributos, especialmente a categoria, que era um enumerador.

A partir da identificação da violação do princípio aberto-fechado ao tentar adicionar atributos únicos a cada produto (ex: Validade), foi necessária a fatoração do código, transformando o produto em uma classe abstrata.

Antes da refatoração:	Após a refatoração:
<pre> Java public class Produto { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long codigo; private BigDecimal preco; @Enumerated(EnumType.STRING) private Categoria categoria; private String nome; private String descricao; public Produto(Long codigo, BigDecimal preco, Categoria categoria, String nome, String descricao) { this.codigo = codigo; this.preco = preco; this.categoria = categoria; this.nome = nome; this.descricao = descricao; } } public enum Categoria { FLORES, FLORES_SECAS, ARRANJOS, ... } </pre>	<pre> Java public abstract class Produto{ ... } public class FlorSeca extends Produto{ private LocalDate validade; @Enumerated(EnumType.STRING) private UnidadeDeMedida unidadeDeMedida; ... } public interface ProdutoFactory { Produto criar(Map<String, Object> dados); String getCategoria(); } public class ArranjoFactory implements ProdutoFactory{ @Override Produto criar(Map<String, Object> dados){ ... } @Override String getCategoria(){ return "ARRANJOS" } } </pre>

Tabela 2 - Refatoração do código após princípio de aberto e fechado.

Grupo 3: Floricultura

Logo, podemos adicionar novas fábricas de produtos e produtos que estendem a classe Produto, sem necessitar alterar classes já existentes, deixando claro o princípio de Aberto-Fechado.

7.4 Substituição de Liskov (SOLID)

O princípio SOLID da Substituição de Liskov (LSP) define que objetos de uma classe filha devem poder ser utilizados no lugar da classe base sem alterar o funcionamento correto do sistema. No módulo de cliente, esse princípio foi aplicado por meio de uma hierarquia de herança que permite tratar clientes Pessoa Física e Pessoa Jurídica de maneira uniforme, reduzindo acoplamento e evitando verificações explícitas de tipo espalhadas pelo código.

Inicialmente, os dois tipos de cliente eram tratados utilizando estruturas condicionais. Sempre que alguma operação precisava ser executada, como carregar o perfil, atualizar informações ou montar respostas da API, o sistema verificava qual era o tipo concreto do objeto para decidir quais campos deveriam ser utilizados.

A refatoração introduziu uma classe abstrata Cliente com os atributos e comportamentos comuns, permitindo que PessoaFisica e PessoaJuridica acrescentem apenas seus campos específicos. Com isso, todas as operações do sistema passaram a ser escritas em termos da abstração, eliminando as verificações condicionais repetitivas e garantindo que novos tipos de cliente possam ser adicionados sem impacto nas camadas superiores.

Antes da refatoração	Após a refatoração
<pre>Java public class Cliente { private String nome; private String email; private String telefone; private String googleId; // dados de pessoa física private String cpf; private LocalDate dataNascimento; // dados de pessoa jurídica private String razaoSocial; private String cnpj; private String tipo; // PF ou PJ }</pre>	<pre>Java public abstract class Cliente { private String nome; private String email; private String telefone; private String googleId; } public class PessoaFisica extends Cliente { private String cpf; private LocalDate dataNascimento; } public class PessoaJuridica extends Cliente { private String razaoSocial; private String cnpj; }</pre>

Grupo 3: Floricultura

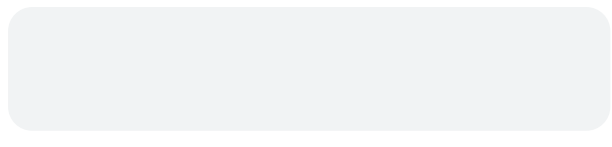
	
---	--

Tabela 3 - Refatoração do código após aplicação do princípio de Substituição de Liskov

7.5. Inversão de Dependência (SOLID)

O princípio SOLID, que consiste em as entidades devem depender de abstrações, não de implementações. Assim, no uso da interface `ProdutoRepository`, estendendo `JpaRepository`. Ao definir métodos customizados através de assinaturas abstratas ou consultas derivadas (como `findByCategoria`), o sistema assegura que a camada de negócio dependa apenas de contratos, e não de tecnologias de persistência específicas. Essa abstração permite que o provedor de banco de dados (MySQL/JPA) seja alterado ou substituído sem causar impactos colaterais nas regras de negócio do `ProdutoService`, demonstrando a eficácia da inversão de controle na manutenibilidade do código.

Java

@Repository

```
public interface ProdutoRepository extends JpaRepository<Produto, Long> {  
    @Query("SELECT p FROM Produto p WHERE TYPE(p)= :categoria")  
    List<Produto> findByCategoria(@Param("categoria") Class<? extends  
    Produto> categoria);  
  
    Optional <Produto> findByName(String nome);  
    List<Produto> findByQuantidadeLessThan(int quantidade);  
}
```

Tabela 4 - Exemplo de repositório contendo inversão de dependência a partir do `JpaRepository`.

8. PADRÕES GOF

8.1. Factory Method para Produtos

O padrão GoF Factory Method é aplicado na camada de serviço de Produto para resolver o gerenciamento de múltiplos tipos de produtos (flor, flor seca, arranjo, buquê, cartão e kit), evitando o uso de estruturas condicionais, como por exemplo:

Java

```
if (categoria.equals("FLORES")) produto = objectMapper.convertValue(dados,  
    Flor.class);
```

Grupo 3: Floricultura

```
else if (categoria.equals("BUQUES")) produto = objectMapper.convertValue(dados,
Buque.class)
```

Tabela 5 - Exemplo de Factory Method

Dessa forma, o projeto fica coeso com os padrões GRASP, como por exemplo, de especialista, com as classes contendo as responsabilidades devidas, bem como princípios SOLID de Substituição Liskov, em que as classes contém métodos e atributos que estão sendo utilizados e mantendo o princípio de aberto-fechado, permitindo que novos produtos possam ser adicionados, sem alterar o código.

Assim, o projeto fica organizado no seguinte formato:

- Produto: Classe abstrata em que múltiplos objetos compartilham características em comum
- Produtos concretos (Arranjo, Buque, Cartao, Flor, FlorSeca, Kit): Extendem a classe produto, possuindo características únicas.
- Criador (ProdutoFactory): Interface que contém a declaração do método de criação (produtoCriar).
- Criadores concretos (ArranjoFactory, BuqueFactory, CartaoFactory, FlorSecaFactory, FlorFactory, KitFactory): Implementam o método de criação, adequando-se a classe que representam.

E por fim, o ProdutoService lida com a lógica da criação dos produtos.

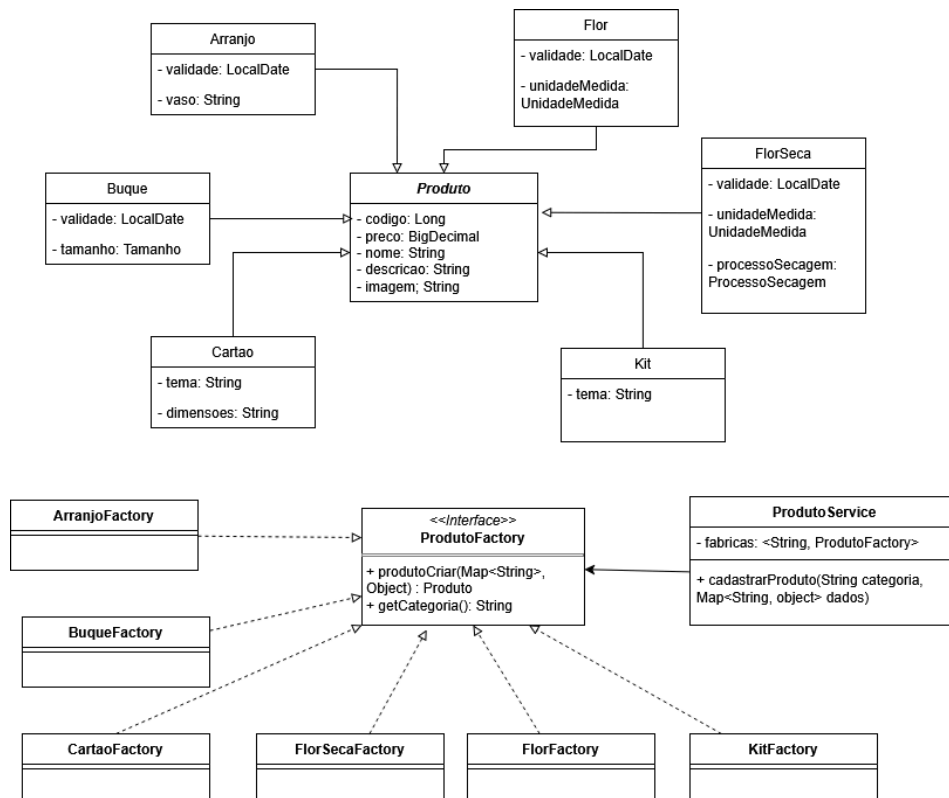


Imagem 18 - Diagrama do padrão Factory Method para a entidade produto

8.2 Observer para Favoritar Produto

O padrão GoF Observer é um padrão de projeto comportamental aplicado na classe Favorito (que possui relacionamento com Produto) para resolver o gerenciamento de alertas automatizados enviados aos clientes da floricultura. Seu papel, por definição, é implementar uma dependência um-para-muitos entre objetos, de maneira que, quando um objeto muda de estado (no caso o produto favoritado), todos os seus dependentes são notificados e atualizados automaticamente.

Na arquitetura do sistema, a implementação deste padrão elimina o acoplamento rígido entre a entidade que sofre a modificação (o Produto) e os elementos que precisam reagir a essa modificação (as listas de favoritos dos múltiplos clientes). Sem o Observer, o sistema seria forçado a utilizar rotinas manuais varrendo o banco de dados em busca de atualizações, o que não configura uma arquitetura limpa e está longe das boas práticas. Com o padrão, sempre que um produto sofre modificações cruciais (como uma redução de preço ou retorno ao estoque), todos os usuários que favoritaram esse item específico recebem a notificação de forma reativa e assíncrona.

Dessa forma, o projeto fica coeso com os padrões GRASP, como o de Baixo Acoplamento e Especialista, garantindo que a entidade Produto ou o ProdutoService não precisem conhecer a identidade ou os detalhes de implementação de quem os está observando, limitando-se apenas a disparar o evento. Também atende aos Princípios SOLID, com destaque para o princípio de Aberto-Fechado, visto que novas regras de negócio (como enviar um e-mail ou outras formas de notificação) podem ser acopladas ao sistema criando novos observadores concretos, sem a necessidade de alterar uma única linha de código da classe principal de produto ou das regras de favoritos já implementadas.

Assim, para implementar o padrão Observer usamos a seguinte estrutura (citando as classes principais):

1. Pasta Observer - Núcleo do Padrão:

- **ProdutoPublisher.java:** Atua como o Publisher. É a classe central que detém o estado monitorado e mantém a lista dinâmica de assinantes (subscribers). Ela expõe o método especialista `notificarAlteracaoProduto(...)` que, simulando a Lógica de Notificação, executa um laço `foreach` sobre todos os ouvintes registrados para avisar que o produto mudou.
- **ProdutoObserver.java:** Corresponde à interface abstrata `Subscriber`. Estabelece o contrato de comunicação, definindo a assinatura do método `atualizar(Produto produto, String mensagem)`. Qualquer classe que pretenda reagir a variações de um produto na plataforma deve assinar este contrato.
- **NotificacaoBancoObserver.java:** É o Concrete Subscribe, o ouvinte real que implementa a interface e executa a ação concreta do ecossistema: quando acionado pelo publicador, ele descobre por meio

Grupo 3: Floricultura

do banco de dados quais clientes específicos favoritaram aquele produto e gera as notificações

2. Camada de Domínio e Persistência (model e repository):

- **Produto (Model) e ProdutoService (Service):** Representam o estado principal manipulado pela aplicação. O método atualizarProduto atua como o gatilho gerador do evento. Assim que o estado do produto é alterado e persistido, o serviço evoca o ProdutoPublisher para iniciar a cadeia de notificações, mapeando o fluxo clássico em que a regra de negócio dispara o aviso aos interessados.
- **Favorito (Model) e FavoritoRepository (Repository):** Componentes especialistas de negócio. O repositório é peça-chave utilizada pelo observador concreto (NotificacaoBancoObserver) para mapear dinamicamente a relação um-para-muitos, localizando instantaneamente quais IDs de usuários possuíam o produto modificado na lista de desejos.
- **Notificacao (Model) e NotificacaoRepository (Repository):** A entidade que materializa a mensagem criada pela reação do padrão (armazenando o texto, data e o booleano lida). O repositório realiza a gravação física dessa resposta gerada pelos observadores.

Além disso, temos DTOs e Mappers, responsáveis por definir e converter a estrutura dos dados trafegados entre backend e frontend, e Controllers, que expõem os endpoints da API e direcionam corretamente o fluxo das requisições. Dessa forma, as notificações geradas pelo Observer podem ser consultadas e atualizadas pela interface do sistema.

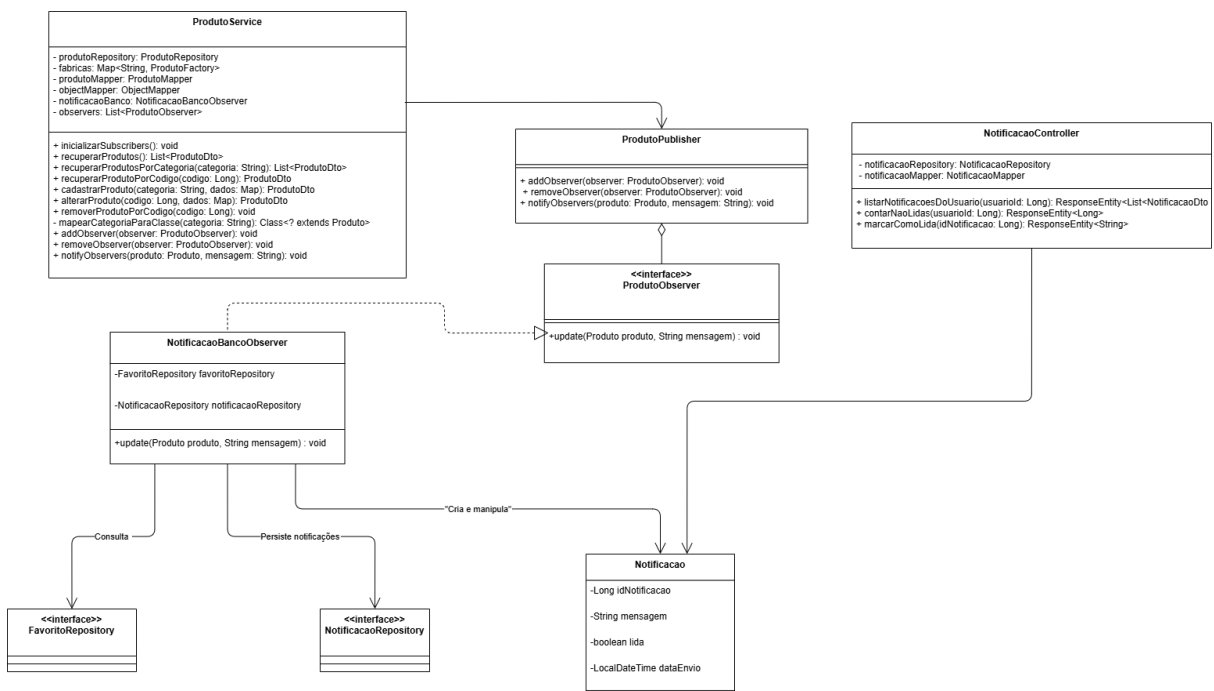


Imagem 19 - Diagrama das classes envolvidas na implementação do Observer

Grupo 3: Floricultura

8.3 Strategy para Precificação dos Planos de Assinatura

O padrão GoF Strategy é aplicado na funcionalidade de Assinaturas para resolver o cálculo de preço dos diferentes planos disponíveis (mensal, quinzenal e semanal), evitando o uso de estruturas condicionais dentro do serviço, a exemplo de:

Java

```
if (tipoPlano.equals("Mensal")) valor = 89.90;  
else if(tipoPlano.equals("Quinzenal")) valor = 149.90;  
else if(tipoPlano.equals("Semanal")) valor = 249.90;
```

Tabela 6 - Exemplo de Strategy

Dessa forma, a regra de precificação de cada plano fica isolada em sua própria classe, deixando o AssinaturaService responsável apenas pelo fluxo de contratação ou alteração de assinatura. Isso mantém o projeto mais coeso e alinhado aos princípios SOLID, principalmente o princípio open-closed, pois novos planos podem ser adicionados sem modificar diretamente a lógica principal do serviço. Com isso, temos:

- **PrecificacaoAssinaturaStrategy**: Interface que define o contrato das estratégias de precificação, contendo os métodos `getTipoPlano` e `calcularPreco`.
- **Estratégias concretas(PrecificacaoMensalStrategy, PrecificacaoQuinzenalStrategy, PrecificacaoSemanalStrategy)**: Implementam a interface e definem o preço específico de cada plano de assinatura.
- **PrecificacaoAssinaturaResolver**: Classe responsável por selecionar a estratégia correta de acordo com o tipo de plano informado.
- **AssinaturaService**: Utiliza o resolver para calcular o valor da assinatura, sem precisar conhecer diretamente as regras de preço de cada plano.

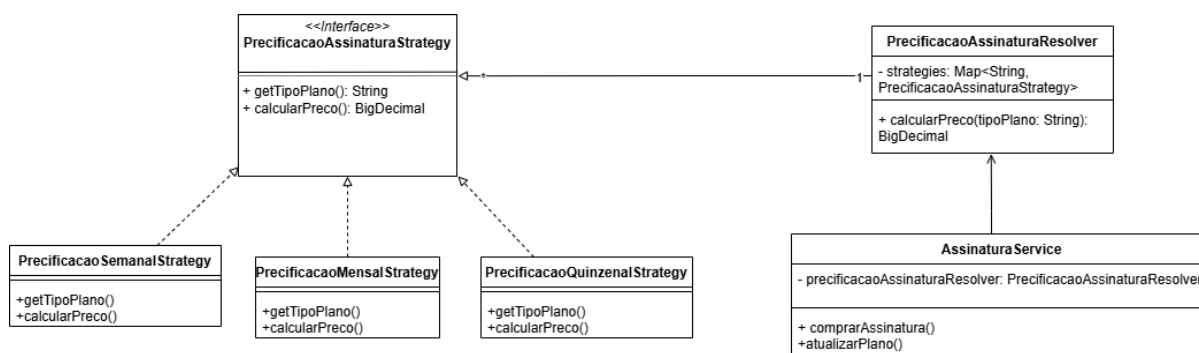


Imagem 20 – Diagrama de classes da assinatura com Strategy aplicado

8.4. State para o ciclo de vida do pedido

O padrão GoF State é aplicado na entidade pedido para gerenciar as diferentes fases pelas quais um pedido passa, eliminando estruturas condicionais complexas e permitindo que

Grupo 3: Floricultura

cada estado encapsule suas próprias regras de negócio. O estado de um pedido seria controlado por meio de verificações como:

```
Java
if (pedido.getStatus().equals("PENDENTE")) {
    pedido.setStatus("PAGO");
} else if (pedido.getStatus().equals("PAGO")) {
    pedido.setStatus("ENVIADO");
} else if (pedido.getStatus().equals("ENVIADO")) {
    pedido.setStatus("ENTREGUE");
} else if (pedido.getStatus().equals("ENTREGUE") ||
pedido.getStatus().equals("CANCELADO")) {
    throw new RuntimeException("Pedido não pode avançar de status");
}
```

Tabela 7 - Exemplo de State

Porém, com a evolução do sistema, essa abordagem torna-se difícil de se manter. O padrão State resolve esse problema usando objetos especializados que representam cada estado possível do objeto. Assim, o padrão passa a ter os seguintes participantes:

- **Contexto (Pedido):** a classe que possui um estado interno e que o comportamento varia conforme esse estado.
- **Estado (EstadoPedido):** interface que declara os métodos correspondentes às ações que podem ser realizadas em um pedido como proximoStatus(), cancelar().
- **Estados Concretos:** implementações da interface para cada fase do ciclo de vida: PedidoPendente, PedidoPago, PedidoEnviado, PedidoEntregue, PedidoCancelado.
- **Serviço (PedidoService):** responsável por gerenciar as mudanças de estado, invocando os métodos apropriados do estado atual sem conhecer detalhes de implementação.

Diagrama:

Grupo 3: Floricultura

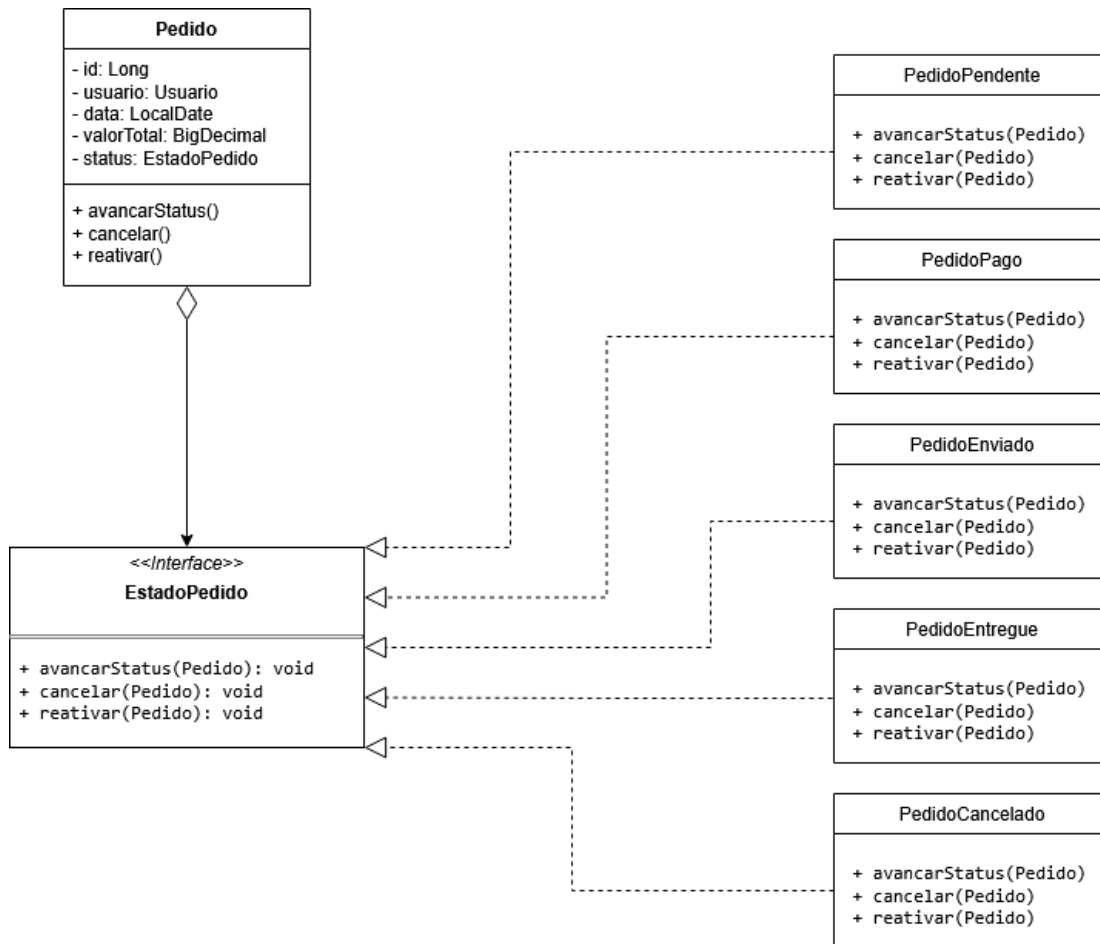


Imagem 21 - Diagrama do Padrão State

8.5 Facade para Dashboard

O padrão GoF Facade é aplicado na camada de serviço do Dashboard para resolver a organização de múltiplos subsistemas (repositórios de pedidos, produtos, mapeadores), evitando que o controlador acumule dependências e regras de agregação, como por exemplo:

```
Java
// sem fachada - o controller vira um "god class"
List<Pedido> pedidos = pedidoRepo.findByDataBetweenCompleto(inicio, fim);
List<PedidoResumoDto> pedidosHoje = pedidoMapper.toPedidosResumo(pedidos);
BigDecimal faturamento = pedidoRepo.faturamentoDia(inicio, fim, statusValidos);
List<Produto> criticos = produtoRepo.findByQuantidadeLessThan(10);
DashboardDto dto = new DashboardDto(pedidosHoje, ..., faturamento, ...);
```

Tabela 8 - Exemplo de Facade

Dessa forma, o projeto fica coeso com os padrões GRASP, como o Especialista (o serviço concentra a informação necessária para montar o DTO) e o Controlador (o

Grupo 3: Floricultura

controlador delega totalmente a lógica de negócio), bem como os princípios SOLID de Responsabilidade Única (cada classe tem um único motivo para mudar) e Aberto-Fechado (novas seções do dashboard podem ser adicionadas sem alterar o controller).

Assim, o projeto fica organizado no seguinte formato:

Fachada (Facade): DashboardService – oferece o método getDashboardHoje() que retorna o DTO consolidado, coordenando os subsistemas.

Cliente DashboardController – utiliza a fachada para obter os dados e devolver a resposta HTTP, sem conhecimento dos detalhes internos.

Subsistemas: PedidoRepository, ProdutoRepository, PedidoMapper,
E por fim, o DashboardService lida com a lógica completa de montagem do DashboardDto, mantendo o restante do sistema simples e desacoplado. Se futuramente for adicionar a lógica da entrega ou a integração de novos subsistemas, só altera o Dashboard Service

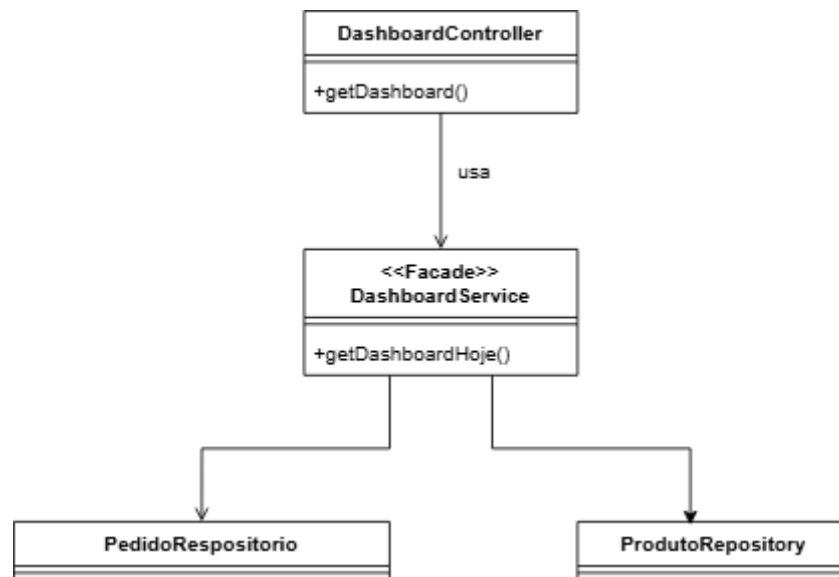


Imagem 22 - Diagrama do Padrão Facade

8.6 Chain of Responsibility para validação do Pedido

O padrão GoF Chain of Responsibility é um padrão de design que permite que requests sejam passadas por uma cadeia de handlers. Quando recebem uma request, cada handler decide entre processar a request ou passá-la para o próximo handler da cadeia.

No projeto, ele é utilizado para validação do pedido. Isto é, para que o pedido seja concluído, é necessário fazer uma série de verificações: como se o produto está disponível, se o pagamento foi aprovado, etc. Para evitar uma série gigante de verificações sequenciais, é utilizado esse design pattern.

- **Interface Handler (Pedido):** declara a interface e é comum a todos os handlers, contém apenas métodos básicos.
- **Handler Base (PedidoValidationHandler):** Classe abstrata que implementa a interface Handler e funciona como base para os handlers concretos.

Grupo 3: Floricultura

- **Handlers Concretos:** contém o código para processar as requests. Quando recebem uma request, decidem se processam ou se passam para o próximo handler.
- **Serviço (PedidoService):** responsável por passar as requests para a cadeia lidar.

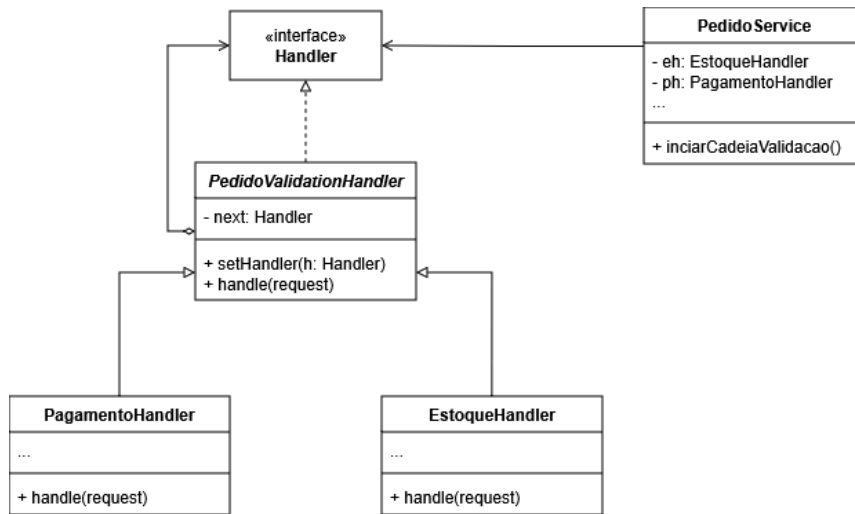


Imagem 23 - Diagrama de classe para

9. LINHA DE PRODUTO DE SOFTWARE (LPS)

Características mandatórias:

- Criar conta (Cliente e Administrador);
- Fazer login em conta (Cliente e Administrador);
- Visualizar catálogo;
- Adicionar ao carrinho;
- Finalizar pedido;
- Visualizar histórico de compras;
- Controle de estoque;
- Buscar produtos por nome;
- Buscar produtos por categoria;
- Aplicar cupom de desconto;
- Gerenciar regras promocionais;

Características opcionais:

- Produtos customizáveis;
- Realizar assinaturas;
- Fazer reserva;
- Kits de presente;
- Favoritar produtos;
- Sistema de notificação;

Grupo 3: Floricultura

Características alternativas:

- Meios de pagamento (Pagar com Pix, Pagar com cartão)

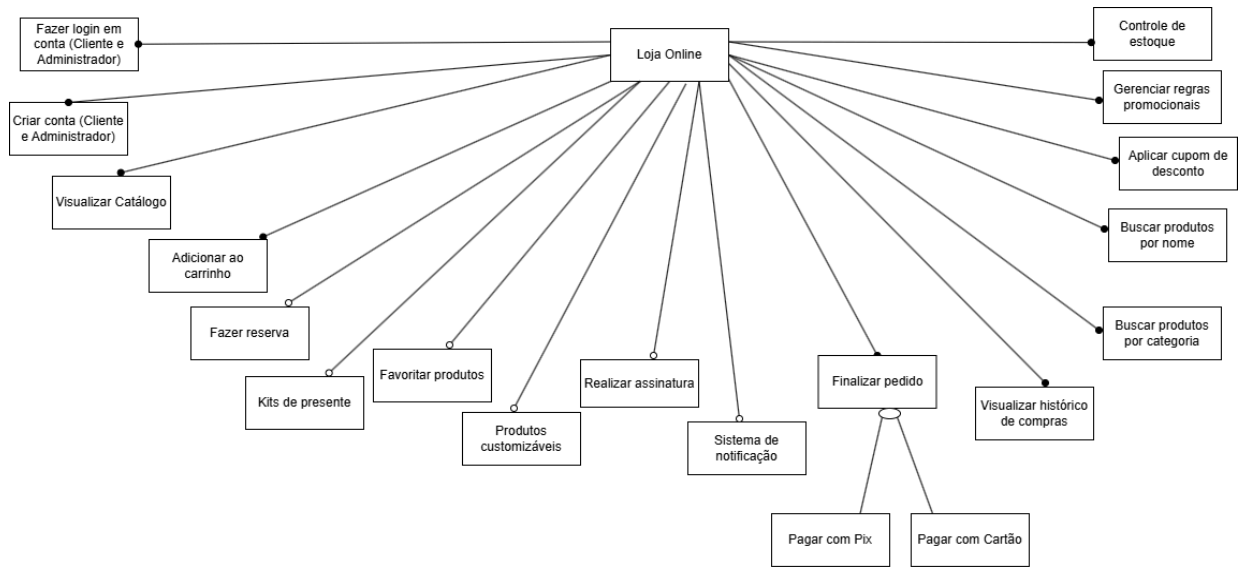


Imagem 24 - Diagrama de análise para pagamentos

10. DESCRIÇÃO DE IMPLEMENTAÇÃO

10.1 Implementações do sistema

10.1.1 Funcionalidades do cliente(front-end)

- Cadastro e login via conta Google(authenticação);
- Catálogo de produtos com filtro por categoria;
- Página de detalhes de um produto com avaliação de clientes;
- Carrinho de compras;
- Finalização de compras com seleção de endereços e aplicação de cupom de desconto;
- Histórico de pedidos para acompanhar status;
- Escolha de assinatura de flores com escolha de plano e gerenciamento (contratar, alterar e cancelar);
- Reserva de decoração de flores para eventos(casamentos, debutantes, corporativos);
- Favoritar produtos (com notificação automática ao retorno ao estoque ou redução de preço);
- Gerenciamento de endereços cadastrados;
- Visualização e envio de avaliações de produtos pós-compra;

Grupo 3: Floricultura

10.1.2 Funcionalidades do administrador (painel de administrador)

- Login de administrador via Google com validação de papel (SUPER_ADMIN, GERENTE)
- Dashboard com resumo do dia: pedidos, faturamento, estoque crítico.
- Crud de produto, cupons e administradores
- Relatório gerencial por período (total de pedidos, faturamento, ticket médio, clientes únicos)
- Gerenciamento de eventos/reservas(aprovar ou negar solicitações)
- Página de histórico de todos os pedidos do sistema, com filtro por data e nome

10.1.3 CRUDs implementados

- **Produto:** O usuário administrador pode adicionar, remover, editar e excluir novos produtos a partir da interface do administrador, que é atualizado nos produtos disponíveis para compra de acordo com sua categoria.
- **Pedido:** O cliente pode fazer um pedido ao finalizar a compra na página do carrinho. Os pedidos realizados ficam disponíveis na página de pedidos do cliente.
- **Carrinho:** O cliente pode adicionar e remover produtos ao carrinho. O cliente também pode visualizar os produtos que estão em seu carrinho.
- **Avaliação:** Os usuários podem avaliar qualquer produto atribuindo uma nota e um comentário exclusivamente em texto, diretamente na página de detalhes do produto relacionado. As avaliações são atribuídas ao usuário logado e aparecem na página de detalhes do produto, mostrando o nome do usuário que publicou, o comentário e a nota atribuída.
- **Reserva:** Os usuários podem solicitar uma reserva na aba “Eventos” para uma data futura adicionando os produtos desejados e fornecendo detalhes do pedido. Depois do envio da solicitação, o usuário fica no aguardo da confirmação da administração, que avalia o pedido e decide se a reserva está confirmada ou recusada. Enquanto isso, o cliente consegue acompanhar o status da reserva em seus pedidos.
- **Usuário Administrador:** Existem 2 tipos de administradores: o gerente e o super_admin, esse segundo é capaz de criar, atualizar, remover ou editar administradores no sistema.
- **Cliente:** O cliente pode se cadastrar, excluir sua própria conta, editar seus dados. Existem dois tipos de cliente: Pessoa Física e Pessoa Jurídica.
- **Assinatura:** A assinatura permite que o cliente receba flores de forma recorrente, escolhendo um plano de entrega conforme sua preferência. Após o pagamento, a assinatura fica vinculada ao perfil do cliente.
- **Cupom:** O CRUD de cupom fica na página do administrador, lá ele pode criar, excluir e visualizar cupons, atribuindo o desconto percentual, nome do cupom, descrição, limite de uso, data de início e data de finalização.
- **Endereço:** O cliente pode cadastrar um novo endereço, pode editá-lo, excluí-lo e ver a lista de endereços associados a sua conta na página de endereços.

10.1.4 Casos de uso transacionais

Grupo 3: Floricultura

Os casos de uso transacionais implementados no projeto foram: **realizar reservas e realizar pedido, realizar assinaturas** .

O caso de uso de realizar pedidos só acontece após o cliente logado adicionar itens no carrinho, que só são possíveis caso tenham ao menos uma unidade disponível no estoque, sinalizada na página de cada produto, bem como no estoque do administrador. Ao adicionar itens no carrinho, o usuário pode alterar a quantidade respeitando os itens disponíveis em estoque.

Quando a compra é confirmada, o banco de dados de produtos (estoque) tem seus valores alterados, bem como o perfil do usuário que mantém um histórico de compras. Caso a compra sofra algum erro durante a transação, a atomicidade da transação é respeitada, ou seja, todo o processo de compra é desfeito e não causa alterações no estoque.

O caso de uso de realizar reservas o usuário precisa acessar o menu de eventos e preencher o formulário selecionando os produtos e fornecendo as informações do evento a ser realizado. Ao finalizar o formulário, o usuário solicita a reserva e aguarda a confirmação do Administrador.

O administrador possui um painel com todas as reservas a serem confirmadas e faz o papel de confirmar a reserva, que altera no estoque os produtos reservados. O cliente consegue ver se sua reserva foi confirmada em “Meus Pedidos”. Caso o administrador não confirme a reserva, o pedido é desfeito por completo, não alterando o estoque.

Por fim, o caso de uso de realizar assinaturas permite que o usuário tenha somente até uma das três assinaturas disponíveis no sistema: Mensal, Quinzenal, Semanal. Que ao iniciar compra exige um formulário com o tipo de flores a serem recebidas, assim, o item é adicionado ao carrinho e segue o padrão do caso de uso de realizar pedido.

Ao pagamento ocorrer sem falhas, a assinatura pode ser cancelada a qualquer momento pelo usuário dentro do seu perfil, bem como alterar o tipo da assinatura. Em caso de falhas a transação é reiniciada.

10.1.5 Caso de uso de listagem do sistema(relatório)

Relatório Gerencial (RelatorioService.gerarRelatorio):

Endpoint GET /relatorios protegido para administradores com papel GERENTE ou SUPER_ADMIN. Recebe um período como parâmetro e retorna: total de pedidos, faturamento agregado, ticket médio, número de clientes únicos e lista de cupons utilizados no período.

10.2 Tecnologias, frameworks e ferramentas utilizadas;

10.2.1 Frameworks

O back-end do projeto foi realizado em Java, utilizando o framework Spring-Boot para facilitar a criação de API rest. Esta API estabelece uma arquitetura cliente-servidor, com

Grupo 3: Floricultura

uma divisão de responsabilidades em front-end (cliente), que atua realizando requisições HTTP (GET, POST, PUT e DELETE) para solicitar ou enviar dados. O back-end processa as requisições, manipulando o banco de dados e retornando no formato JSON.

Enquanto para o front-end foi utilizado o React, focado para a interface gráfica dos usuários, a partir de peças individuais reutilizáveis (componentes), que evita que o código seja reescrito e facilitando a organização. Além disso, o Tailwind-css foi integrado com o React, permitindo a estilização das páginas WEB, evitando o uso de longos arquivos .css, o Tailwind é utilizado a partir do atributo classname em cada tag do HTML.

Para a integração para a autenticação com o Google foi utilizado o firebase que oferece o serviço Firebase Authentication integrado com o google, permitindo fazer o uso de contas do google para criação de contas de usuários administradores e clientes.

10.2.2 Ferramentas de desenvolvimento

MySQL Workbench para persistência de dados

Git hub para controle de versões e colaboração

IntelliJ IDEA / VS Code como IDE

Google docs para relatório

10.3 Arquitetura geral da solução

A arquitetura da solução se manteve a mesma, utilizando arquitetura MVC (Model-View-Controller) e Cliente-Servidor para organizar as responsabilidades do projeto. No spring boot, o padrão MVC é implementado seguindo as lógicas de classes de Repository, que faz comunicação direta com o banco de dados MySQL, Service, que reside a lógica de negócio do sistema, e por fim Controller, que recebe as requisições HTTP do cliente e redireciona as ações. Enquanto o papel da View do MVC é gerenciado pelo React e Tailwind-CSS que constroem a interface gráfica que o usuário interage.

A arquitetura cliente servidor separa as responsabilidades em cliente e servidor a partir do protocolo HTTP, utilizando métodos de GET, POST, PUT, DELETE, em que o cliente dispara essas requisições ao clicar em botões ou carregar páginas. Assim, o back-end do sistema atua como servidor, que lida com as requisições.